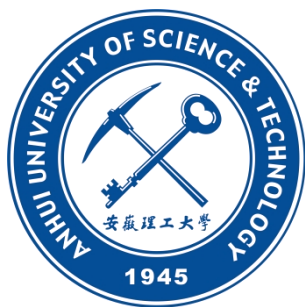


安徽理工大学

ANHUI UNIVERSITY OF SCIENCE & TECHNOLOGY

本科毕业设计说明书

(2026 届)



中文题目： 基于图像增强的恶劣天气车辆感知系统设计与实现

英文题目： Vehicle Perception System for Adverse Weather Based on
Image Enhancement

学 院：	计算机科学与工程学院		
专 业：	计算机科学与技术		
班 级：	计算机 22-5 班		
学 号：	2022304393		
姓 名：	刘武		
指导教师：	许欢	职称	中级
企业导师：	马维士	职称	副高级

2026 年 5 月

摘要

智能交通与自动驾驶技术发展得越来越快，车辆感知技术也成了这个领域里的重点研究内容，而雾天、雨天、低光照这类恶劣天气，会让拍摄的图像质量变差，导致传统检测算法没法提取到有效特征，最终让检测效果出现明显下滑，针对这个实际问题，该项目搭建出分析不同模型增强技术的恶劣天气车辆感知系统，用它来改善恶劣环境下车辆检测精度偏低的现状。

本系统把 YOLOv8 模型当作核心检测结构，搭配了多种图像优化手段，用 NAFNet 网络完成图像去雾去雨操作，把画面原有的细节还原回来，靠 MBLEN 算法优化低光照画面，提亮暗处画面的同时凸显出细节内容，另外会把 CLAHE 和形态学处理方式当作备用方案，保障特殊场景下的处理效果，系统内置的场景识别功能，能依托多项图像质量参数自主匹配对应的优化策略，不用额外训练专属模型，整体运算负担也更轻。

本系统采用前后端分离的开发架构，依靠推理优化工具把 CPU 设备的运行效率提了上去，能适配图片、视频、实时摄像三种识别模式，还能精准识别五类常用车辆，也可以借助数据库把检测数据长久留存下来。

关键词：图像增强；YOLOv8；深度学习；车辆检测；恶劣天气

ABSTRACT

With the rapid development of intelligent transportation and autonomous driving technologies, vehicle perception technology has become a key research focus in this field. Adverse weather such as fog, rain, and low light conditions can degrade image quality, preventing traditional detection algorithms from extracting effective features and ultimately leading to a significant decline in detection performance. To address this issue, the project has built a severe weather vehicle perception system that analyzes different model enhancement technologies, using it to address the current low detection accuracy in harsh environments.

This system uses the YOLOv8 model as its core detection structure, combined with various image optimization methods. It uses the NAFNet network to perform image dehazing and defrosting, restoring original image details. The MBLLEN algorithm optimizes low-light images, brightening dark areas while highlighting detailed content. Additionally, CLAHE and morphology processing methods are treated as backup solutions to ensure processing performance in special scenarios. The system has built-in scene recognition functions. It can autonomously match optimization strategies based on multiple image quality parameters, eliminating the need for additional dedicated model training and reducing overall computational load.

This system adopts a front-end and back-end separation development architecture, relying on inference optimization tools to improve CPU device efficiency. It can adapt to three recognition modes: image, video, and real-time camera, accurately identify five common vehicle categories, and retain detection data long-term through databases.

Keywords: image enhancement, yolov8, deep learning, vehicle detection, adverse weather

目录

摘要	I
ABSTRACT	II
1. 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 车辆检测技术研究现状	2
1.2.2 图像增强技术研究现状	3
1.2.3 恶劣天气车辆感知研究现状	4
1.3 主要研究内容	4
1.4 论文组织结构	5
2. 需求分析与可行性分析	6
2.1 系统设计目标	6
2.2 需求分析	6
2.2.1 功能需求分析	6
2.2.2 非功能需求分析	6
2.3 可行性分析	7
3. 关键技术与核心算法	10
3.1 技术选型分析	10
3.1.1 深度学习框架选型	10
3.1.2 后端框架选型	10
3.1.3 数据库选型	10
3.1.4 前端技术选型	10
3.1.5 ONNX Runtime 推理加速方案	11
3.1.6 技术选型对比总结	11
3.2 图像增强算法	12
3.2.1 图像增强模块整体架构	12
3.2.2 基于 NAFNet 的去雾与去雨增强	13
3.2.3 基于 MBLEN 的低光照增强网络	15
3.2.4 基于 CLAHE 和形态学操作的传统增强方法	17
3.3 自动场景识别算法	18
3.3.1 算法设计与原理	18

3.3.2	阈值确定与优化	20
3.4	基于 YOLOv8 的车辆检测算法	20
3.4.1	YOLOv8 模型概述	20
3.4.2	车辆检测器的实现	21
3.4.3	模型规格与性能权衡	22
3.5	ONNX Runtime 推理优化	23
3.5.1	多模型管理方案	23
3.5.2	Zero-DCE 低光照增强的 ONNX 实现	24
3.6	性能评测系统	25
3.6.1	评估指标体系	25
3.6.2	评测指标实现	25
3.6.3	评测流程设计	28
4.	系统设计与实现	30
4.1	系统总体架构	30
4.1.1	架构设计原则	30
4.1.2	系统分层架构	30
4.1.3	系统模块划分	31
4.1.4	模块间相互关系	32
4.1.5	数据处理流程设计	33
4.2	前端设计与实现	34
4.2.1	页面结构设计	34
4.2.2	图片识别模块	35
4.2.3	视频识别模块	35
4.2.4	摄像头实时识别模块	36
4.2.5	历史记录模块	36
4.2.6	测试结果分析页面	37
4.3	后端路由设计与实现	38
4.3.1	图像处理路由	39
4.3.2	视频处理路由	39
4.3.3	摄像头实时检测路由	40
4.4	数据库设计	40
4.4.1	数据库选型与连接管理	40
4.4.2	数据库 E-R 图	40

4.4.3 数据库数据流图	41
4.4.4 数据表结构	42
4.4.5 数据库操作封装	43
4.5 视频流处理方案	44
4.6 图像增强服务的多模型管理	44
4.7 系统安全性设计	45
5. 系统测试与结果分析	46
5.1 测试环境	46
5.2 测试数据集	46
5.3 评估指标体系	46
5.4 基准测试流程	47
5.5 检测性能分析	47
5.5.1 雾天条件	47
5.5.2 雨天条件	48
5.5.3 低光照条件	49
5.6 图像质量分析	50
5.7 处理时间分析	51
5.8 实验结论	51
6. 总结与展望	53
6.1 工作总结	53
6.2 系统特色与创新点	53
6.3 存在的不足	54
6.4 未来展望	54
结论	56
参考文献	56
致谢	60

1. 绪论

1.1 研究背景与意义

最近十几年，城市规模越来越大，私家车数量也一直在涨。交通拥堵、事故频发这些问题越来越突出，智能交通系统（Intelligent Transportation System, ITS）慢慢成了解决交通问题的一个重要方向。ITS 通过摄像头、传感器这些设备来实时监测道路状况，为交通管理提供数据支撑。根据公安部交通管理局发布的数据，到 2025 年底，全国机动车保有量达到了 4.5 亿辆，其中汽车 3.8 亿辆，车辆基数不断扩大，靠人工来监控交通显然不现实。

智能交通系统的核心能力之一就是车辆检测——即从道路图像或者视频里把车辆识别出来，确定它们的位置和类型^[4]。近几年深度学习技术发展很快，尤其是目标检测领域，新兴算法不断涌现。从 R-CNN，到 Fast R-CNN、Faster R-CNN，再到 YOLO 系列和 SSD，检测的准确率和速度都在不断提高^[1]。YOLO 系列因为模型结构相对简洁、推理速度快，成为工业界应用最广的框架之一^[2]。本系统选用的是 2023 年 Ultralytics 发布的 YOLOv8 版本，此版本去掉了预设锚点框（Anchor-Free），把分类和回归分开处理（Decoupled Head），引入了 C2f 模块。使 YOLOv8 在 COCO 数据集上的综合表现良好，亦因此本系统选此作为核心检测模型。

现如今市面上现存的各类检测模型，大多都在晴朗无云、光照充足这类理想环境中完成训练与实测的工作，现实道路通行场景里却常会出现起雾降雨以及夜间光照不足等各类复杂状况，一方面雾气弥漫的环境中，空气中漂浮的细小颗粒能够把光线进行散射处理，拍摄得到的画面会呈现出偏白的状态，画面本身拥有的清晰程度也会有所下降，车辆实体还容易和周边环境融为一体难以区分，另一方面降雨天气里，飘落的雨水会附着在拍摄设备镜头表层，路面留存的积水也会反射外界光源，进而让采集到的画面产生大量杂乱干扰像素，还有夜间通行时段整体光照强度偏低，整体画面会处于偏暗的状态，画面当中不少细微内容都难以清晰分辨，相关研究数据能够证实，各类恶劣天气能够给目标检测的精准程度带来极大的负面影响，处于雾气浓度适中的环境之下，正常天气里数值维持在 0.95 上下的 YOLOv8 模型 mAP50 指标，会被降到仅有 0.7 左右的水平，降幅已经超出了百分之二十五，出现的这类性能下滑情况，在格外注重运行安全的道路监测以及智能行车应用场景中，是完全无法被认可接纳的。

针对以上问题，本系统首先对图像增强处理，提高图像质量，再用增强后的图像去做检测。图像增强的方法也不少。传统的方法像直方图均衡化、CLAHE、Gamma 校正这些，跑得快但效果有限。基于物理模型的方法比如暗通道先验(DarkChannelPrior, DCP)去雾算法^[4]，在均匀雾天效果不错，但天气稍微复杂点就不太行了。这几年深度学习的

方法进步很大，像 MBLLEN 做低光照增强、Zero-DCE 不需要配对数据就能做增强、NAFNet 结构简单但在去雾去雨上都表现挺好。不过这些方法也有自己的问题：需要一定的硬件算力，而且一个模型一般只能处理一种天气类型。

由此在实际落地运用的过程中能够遇到不少现实难题，一是各类不同的恶劣气候环境该搭配选用何种画面优化手段，二是该怎样在优化成效与运行速率之间找到合适的平衡状态，三是能否让运行程序自主判别当下所处的天气类别并自行匹配对应的画质优化方案，面对这类现存的实际难题，本次研究项目准备搭建出一套可适配恶劣气候环境的车辆识别感知体系，该体系把 YOLOv8 框架当作核心识别架构，同时整合了多种实用的图像优化手段，还增设出可以自主分辨天气状况的功能模块，能够妥善适配雾气天气阴雨天气以及暗光环境这三类高频出现的场景，整套体系还被搭建成为网页使用程序，简易便捷的操作模式也让实际使用变得更为顺畅，若是该套实践方案可以顺利投入实际运行，便能把复杂气候环境里智能交通管控体系具备的稳定运行能力得到有效提升，也能为此类相关领域的实际应用带去相应的助力作用。

1.2 国内外研究现状

1.2.1 车辆检测技术研究现状

车辆检测相关研究工作，已经开展了很长一段时间，早期研究阶段，相关人员都会依靠人工方式完成特征设计工作，其中包含方向梯度直方图、尺度不变特征变换以及类哈尔特征等常见类型，研究人员还会搭配支持向量机、自适应提升算法完成后续分类操作；这类传统检测手段，能够在环境单一的场景中呈现出不错的使用效果，一旦现实道路环境变得繁杂，外界光照出现明显变化，或是道路行驶车辆出现相互遮挡的情况，整套检测方式的识别精准程度就会出现明显下降。

2014 年有业内研究人员研制出了 R-CNN 检测算法，此种算法能够依靠卷积网络完成图像特征的提取工作，车辆目标具备的识别准确率也得到了显著提高，一方面该类算法有着十分突出的使用缺陷，偏低的运行处理速度成为主要问题，系统还需要对众多预选区域逐个开展特征提取的相关运算，单张影像的全部处理流程会消耗掉不少时长，现场即时性识别的使用标准也就难以顺利达成，另一方面后续改良而来的 Fast R-CNN 算法融入了感兴趣区域池化的处理形式，各类预选区域都能被对应映射到特征影像之内完成池化操作，得到有效加快的还有整套算法的运转效率，在原有基础上再度优化而成的 Faster R-CNN 算法搭建起专属的区域生成网络，依靠网络本身就能自主生成所需预选区域，端到端形式的网络训练以及实际推算流程也都得以顺利实现，这类分两个流程完成识别工作的算法架构，虽能拥有不错的识别精准程度，却依旧没办法契合道路交通实时监测这类对运行效率有着严苛标准的应用场景。

以单阶段识别思路打造而成的检测模型运用了全新的架构理念，预选区域生成的前

置流程会被直接省去，原始影像能够被直接用来完成目标方位确定以及种类划分的相关工作，2016 年正式面世的 YOLO 识别模型便是该类算法里极具代表的存在，车辆识别相关任务能够被该模型转化为回归计算任务，单一网络结构可以同步输出目标所在方位与所属类别信息，得到大幅提升的还有整体流程的识别效率，同一阶段推出的 SSD 算法则选取不同尺寸的特征影像开展预判计算，距离较远的小型车辆这类细微目标也能被更加平稳精准地识别出来，经历过多轮版本更迭升级的 YOLO 系列模型，从二代逐步更新迭代到八代版本，内部网络构造与实际训练运用的相关方式也都被不断调整优化。

本次研究过程当中所选用的第八代 YOLO 模型，正式面世于二零二三年，这款全新模型在整体架构上做出了多项实质性调整优化，模型摒弃了传统锚点框设定模式，采用无锚点运行机制，工作人员无需提前设定固定锚点框架，系统可以直接依托特征图像推算出目标物体的中心坐标以及长宽数值，也让后续的数据整理处理流程变得更加简便；该模型还将类别判定与位置回归两项工作划分至相互独立的运行模块当中，有效减少两项工作同步运行产生的相互影响；与此同时，模型主干网络内部增设全新 C2f 功能组件，对比以往常用的 C3 组件，全新组件能够在缩减内部参数数量的基础上，维持同等水平的特征提取实际能力；从实际试验得出的各项数据能够看出，第八代超大规格 YOLO 模型，在天气状况良好的环境里，平均检测精度数值可以达到 0.9851，和当下市面普遍使用的同类检测模型对比，整体实际运用表现十分突出。

国内也有很多研究者做了相关的工作。有的人对 YOLOv4 做了改进，加入注意力机制和特征金字塔。模型在复杂交通场景下效果有提升。有些研究者专门针对远处小车的检测做了多尺度特征融合。总体来看，车辆检测这个方向已经比较成熟。但是在恶劣天气下车辆检测的鲁棒性还有不少的提升空间。

1.2.2 图像增强技术研究现状

图像增强手段可分为传统算法、物理模型与深度学习三类，一方面传统方法里的直方图均衡化能够拓宽灰度分布来提高画面对比度，只是整体处理的模式容易造成画面明暗差异进一步拉大，改良后的 CLAHE 会把画面分成小块单独调节，还能控制对比幅度以防噪点加重，伽马校正可借助对应数值灵活调整画面明暗，操作便捷却需要手动设定参数，依托视觉原理的 Retinex 算法会拆分画面光影结构，通过调整光影部分实现画质优化，各类传统增强方式有着运行高效、无需依赖训练模型的优势，却很难适配内容繁杂的场景，所能呈现出的优化效果也存在一定局限。

物理模型方法里，暗通道先验（DCP）去雾是最有名的。它基于这样一个观察：在户外不含雾的清晰图像中，任意一个小区域里至少有一个颜色通道的值很接近 0。利用这个规律，结合大气散射模型 $I(x)=J(x)t(x)+A(1-t(x))$ 就能估算出透射率 $t(x)$ 和大气光值 A ，把清晰图像恢复出来。这个方法在比较均匀的雾天效果不错，但如果画面里有很

多天空或者白色物体，颜色容易失真。另外它假设每个像素的透射率是局部均匀的，遇到浓淡不均的雾就不太灵了。

深度学习的方法这几年进步很快。在低光照增强方面，MBLLEN 设计了多分支结构，从不同感受野提取特征再融合，能把暗部的细节和亮度都恢复得不错。Zero-DCE 提出了零参考学习的思路，不需要配对数据，训练时用一组设计的损失函数来约束，通过估计一条像素级的亮度增强曲线来调整图像。在去雾方面，FFA-Net 用了通道注意力、像素注意力和特征融合，让网络能重点关注雾浓的区域。去雨方面，PReNet 用递归的方式逐步去掉雨线，每一步的输出再作为下一步的输入。NAFNet 的思路不太一样——很多恢复网络里大量用 ReLU、Sigmoid 等激活函数，NAFNet 发现去掉这些非线性激活函数，改用简单的门控机制，结构更简单但效果不差，部署起来也更友好。不过深度学习模型也有限制：一个模型一般只针对一种退化类型设计的，而且跑起来确实比传统方法吃资源。

1.2.3 恶劣天气车辆感知研究现状

专门研究恶劣天气下车辆检测的工作最近几年才多了起来，大概有以下几个技术方向。

第一条是域自适应。基本想法是，让模型在合成雾天数据和真实雾天数据之间做特征对齐，缩小两者之间的差距。Li 等人搞了个 Domain Adaptive Faster R-CNN，用对抗训练来做跨域对齐，雾天检测精度确实提上去了。Hnewa 等人也做了多尺度的域自适应 YOLO。不过这类方法需要目标域的数据参与训练，实际用起来标注成本比较高。

第二条是增强+检测的联合框架。就是先把图像增强，再送进检测。Tran 等人用 GAN 来做去雨再检测，去雨的同时尽量保留车辆细节。这种方案的好处是可以灵活组合不同的增强模型和检测模型。但到底选哪种增强策略检测效果最好，目前还没有统一的结论。

第三条是数据增强思路。在训练的时候往数据里加模拟的恶劣天气样本，让模型提前“见过”这类图像。可以用物理渲染引擎生成，也可以用图像翻译模型生成。

综合来看，现有的研究大多只关注一种天气类型，很少有人做一个统一的方案把雾天、雨天、低光照都覆盖进来。而且增强策略通常是写死的，不会根据输入图像的实际质量动态调整。这也是本文工作的出发点：做一个能自动识别天气类型、选择增强方法的多策略方案，并把它整合成一个完整的系统来测试。

1.3 主要研究内容

本系统的工作主要围绕五个方面展开，一是多策略图像增强模块，集成了 CLAHE、MBLLEN 等多种增强方法，每种方法都适配不同天气场景，还设计了统一调用接口，不管底层用哪种算法，上层调用方式都一样，而且要是某个深度学习模型文件缺失或加载失败，系统会自动切换到传统方法，不会直接报错停运；二是自动场景识别，这个模块

会根据输入图像的亮度、对比度等质量特征，判断出当前的天气情况，再自动匹配对应的增强策略，判断规则靠统计分析得出的阈值，不用额外训练分类模型，计算量也很小；三是 YOLOv8 检测模型部署，对比过 YOLOv8 不同规格在精度、速度和大小上的差异，选了合适的版本进行部署，还用 ONNX Runtime 做推理加速，让模型在 CPU 上也能正常运行，另外视频和摄像头模式下，会用基于 IoU 匹配的帧间跟踪，避免同一辆车被重复计数；四是 Web 系统开发，后端用 Flask 框架，前端用原生 HTML/CSS/JS，支持图片、视频上传检测和摄像头实时检测，数据库用 MySQL，会保存每次检测的详细记录，方便用户查看历史；五是基准测试和性能评估，在雾天、雨天、低光照和正常天气四类数据集上做了全面测试，从 mAP、PSNR 等多个角度对比不同增强方法的表现，看看增强对检测到底有多大帮助。

1.4 论文组织结构

全文总共分为六个章节，具体内容规划如下，一是绪论章节，能够阐述该项研究的开展意义，梳理车辆识别与图像优化领域的现有研究成果，分析复杂天气下车辆识别存在的各类难题，同时明确全文的研究核心内容，二是系统整体设计章节，可介绍平台整体搭建框架，讲明各个功能模块的配合模式，说明相关技术的选用依据以及前后端各自承担的工作内容，三是核心技术相关章节，会细致讲解场景判定方式、各类图像优化手段、目标检测流程以及评价标准，还会结合实际代码内容展开相关分析，四是系统设计与实现章节，主要介绍后台运行逻辑、数据存储格式、页面布局样式以及视频数据的处理方式等具体搭建细节，五是系统测试与数据分析章节，会标明测试所用设备条件、实验数据来源以及评判标准，展示不同天气环境下的实测数据并完成对比分析，六是总结与展望章节，能够归纳整套系统完成的研究内容与实际作用，点明现存的不足之处，同时规划出后续可以优化完善的发展方向。

2. 需求分析与可行性分析

2.1 系统设计目标

这套系统的整体设计目标，是打造一套能在恶劣天气下高效、精准识别车辆的智能感知平台，具体有五个方面：一是能适配雾天、雨天、低光照三类恶劣天气，通过针对性图像增强改善识别效果；二是在恶劣天气下，车辆检测精度尽量接近正常天气，增强后 mAP50 指标有明显提升；三是具备实时处理能力，单张图像从增强到检测的耗时控制在可接受范围，满足实际使用需求；四是支持图片上传、视频分析、摄像头实时识别三种输入模式，覆盖不同场景；五是有直观的 Web 界面，检测一键操作、结果可视化，还能查询管理历史记录。

2.2 需求分析

2.2.1 功能需求分析

结合上面提到的设计目标，系统得具备以下这些核心功能模块，一是图像上传与管理功能，用户能通过 Web 页面把要检测的道路图像上传上去，系统会自动接收并存储这些上传的文件，还支持 JPG、PNG、BMP 等常见的图像格式；二是自动场景识别功能，系统能自动分析输入图像的视觉质量，还能识别出当前的天气条件类型，像正常天气、雾天、雨天、低光照这些都能识别到，这样就能为后续选择合适的增强策略提供依据；三是图像增强处理功能，系统会根据场景识别的结果自动挑选合适的增强算法，也能让用户手动指定增强方式，对退化的图像进行恢复和增强处理，支持的增强方式有去雾增强、去雨增强、低光照增强还有自动模式；四是车辆检测功能，系统能对增强后的图像进行车辆目标检测，把图像里的轿车、卡车、公交车、摩托车、自行车这五类车辆目标都识别出来，还会输出每辆车的类别标签和置信度分数；五是检测结果可视化与统计功能，系统会在图像上画出检测边界框和标签，还能用直观的方式展示出各类别车辆的数量统计信息；六是模式检测支持功能，系统支持三种输入模式，其中图片模式能支持单张图像上传检测，视频模式能支持对视频文件进行逐帧处理，摄像头模式则能通过 RTSP 协议接入监控摄像头，实现实时检测；七是历史记录管理功能，系统会自动保存每次检测的原始图像、增强后图像、标注后图像还有检测统计后的结果，同时也支持对历史记录进行查询、查看详情和删除的操作。

2.2.2 非功能需求分析

非功能需求分析，一是性能方面的需求，单张图像的完整处理流程，包括上传、场景识别、增强、检测还有结果保存，得在数秒内就完成，视频流的处理也要达到能正常使用的帧率水平；二是可用性方面的需求，系统要提供简洁明了的用户界面，操作流程

也得直观好懂，用户不用具备专业知识，就能把所有检测操作都完成；三是可靠性方面的需求，系统得有完善的错误处理办法，遇到无效输入、模型加载失败、文件读写异常这些情况时，能给出明确的错误提示，确保系统能稳定运行；四是可扩展性方面的需求，系统架构要做好模块化设计，这样后续就能方便地增加新的增强算法、检测模型，或者新增对其他天气类型的支持；五是兼容性方面的需求，系统后端要能兼容 Windows、Linux 这些主流操作系统，前端也要能兼容 Chrome、Firefox、Edge 等常见的主流浏览器。

2.3 可行性分析

2.3.1 技术可行性

技术可行性主要用于核查系统选用的各项核心技术，能否在理论研究和实际应用中具备完备的成熟支撑条件，该系统的整体技术选型已经经过充分调研与细致论证，能够为所有核心技术的落地应用，筑牢稳定可靠的实现基础。

一是在深度学习算法层面，系统搭载的图像增强与车辆检测模型，均是经过学术界大量实践验证的成熟方案，实际应用效果稳定可靠，拥有极高的落地实用性，NAFNet 模型凭借精简且高效的架构设计，能够在图像去雾、去雨任务中，呈现出出色的画面修复效果，MBLLEN 与 Zero-DCE 作为低光照图像增强领域的主流算法，无需依托任何额外的参考素材，就能有效提亮整体画面画质，改善暗光场景下的图像效果，YOLOv8 是现阶段目标检测领域的热门应用方案，可在车辆检测这类通用场景中，平衡好检测精度与运行速度两大核心指标，上述模型均配有公开可用的预训练权重，能够极大简化系统的开发搭建流程，稳步提升整体开发效率^[15]。

二是在开发框架层面，Python 语言搭建了内容丰富、适配性强的算法生态，可全方位满足深度学习的各类开发需求，PyTorch 框架作为当下深度学习训练环节的主流工具，配备了完整的官方文档和活跃的社区交流渠道，能够为模型训练工作提供持续且充足的技术支撑，ONNX Runtime 推理引擎可以把训练完成的模型文件统一转换为适配多平台的格式，彻底打破特定框架运行环境的限制，Flask 后端框架主打轻量化设计，十分适配该系统这类中等规模的服务型应用场景，MySQL 数据库作为行业内普及度极高的关系型数据库，能够为系统的数据持久化存储工作，提供稳定、高效的性能保障。

三是在硬件环境层面，系统对运行设备的配置要求并不苛刻，整体处于合理适配范围，模型核心推理工作能够在普通 CPU 环境下稳定运行，ONNX Runtime 自带的多线程并行优化机制，能够充分调动多核处理器的算力资源，即便使用未搭载独立显卡的常规办公设备，系统也能输出符合日常使用标准的响应速度，大幅降低了系统部署落地的硬件门槛与实施难度。

结合上述多个维度的综合分析能够发现，该系统用到的全部关键技术都已具备成熟

的落地应用条件，技术层面潜在的风险能够得到有效管控，整体技术体系稳定可靠，足以充分保障系统的技术可行性。

2.3.2 经济可行性

经济可行性分析主要围绕开发成本和运行成本两大维度展开综合评估，以此判断项目在实际应用中，能否形成科学合理的投入产出结构。

第一点为开发成本层面，该系统的整套技术方案均依托开源免费工具搭建，PyTorch、ONNX Runtime、Flask、MySQL、OpenCV 等核心开发组件，都可免费获取并应用于商业及学术场景，全程无需支付软件授权费用，各类预训练模型权重均源自公开的学术资源平台，能够规避模型从零训练带来的大量算力消耗与时间成本，开发所用的 Python 语言以及配套的海量第三方开源库，同样均为免费可用资源，整套技术体系的搭建不会产生额外的软件采购费用。

第二点为硬件成本层面，系统在设计初期就充分适配了普通 CPU 运行环境，无需搭载高性能独立显卡等高价硬件设备，配置达标后的常规服务器或个人电脑，就可以满足系统部署、运行的基础需求，针对高并发、大流量的应用场景，可通过新增服务节点、完成水平拓展的方式提升系统整体吞吐量，这种灵活的部署模式，能够让硬件投入随实际使用规模灵活调整，有效控制硬件端的投入成本。

第三点为运维成本层面，系统整体采用 B/S 架构设计，各类终端设备仅需通过浏览器即可完成所有操作，无需安装专属客户端软件，极大减少了后期设备维护的工作量，同时降低了用户的操作培训成本，前后端分离的架构设计，能够让系统各功能模块独立完成更新迭代，有效简化了整体运维流程，降低了运维工作的整体难度。

综合整体情况来看，该系统全生命周期的经济投入成本相对低廉，而系统能够带来的车辆识别效率提升、实际落地应用价值，要远高于前期开发与后期运维的整体投入，项目的经济可行性完全成立，具备良好的落地价值。

2.3.3 操作可行性

操作可行性主要用来核验，无专业技术背景的普通使用者能否顺畅完成系统的各项操作任务，该系统已经针对人机交互逻辑、整体使用流程完成全方位的优化设计，大幅提升了整体易用性。

一方面在界面设计上，系统前端采用简约清晰的布局样式，图片上传、视频解析、摄像头实时识别等核心功能，均在首页设置了醒目直观的导航入口，使用者登录系统后，能够快速定位并调用所需功能模块，检测结果会通过目标框标注、数据统计图表的形式做可视化展示，各类车辆的识别数量、分布占比等关键信息清晰直观，历史数据管理模块采用列表式呈现方式，支持使用者按照时间顺序，精准查阅所有过往的检测记录。

另一方面在使用流程上，系统将完整的检测链路整合为一键式智能操作，使用者只

需完成图像或视频文件的上传步骤，后续的场景识别、图像增强、车辆检测等一系列流程，都会由系统自动循序完成，全程无需手动调整参数或选择算法模型，与此同时，系统也为有高阶使用需求的使用者，预留了手动选择图像增强方式的权限，这种自动化基础操作搭配定制化手动操作的设计模式，能够适配不同操作基础、不同使用需求的各类使用者。

除此之外在学习成本上，系统的所有操作逻辑都贴合主流 Web 应用的使用规范，文件上传依托系统标准的文件选择窗口完成，检测结果直接展示在页面对应区域，历史记录只需点击对应列表条目即可查看完整详情，这类交互方式都贴合大众日常的网络使用习惯，没有计算机专业基础的普通使用者，经过简单的上手摸索，就可以熟练掌握系统全部功能的使用方法。

综合以上内容能够看出，该系统拥有良好的易用性与适配性，各类使用者都能以极低的学习成本快速上手操作，系统的操作可行性得到充分验证

3. 关键技术与核心算法

3.1 技术选型分析

3.1.1 深度学习框架选型

系统的核心算法实现包含了图像增强与车辆检测模型的部署运行，也需要选用适配的深度学习框架，经过多方面的对比后，系统选用了 PyTorch 作为模型训练的框架，同时采用 ONNX Runtime 作为模型部署推理的引擎，选用该引擎的优势主要有两个方面，一是具备良好的跨框架兼容性，能把 PyTorch 训练好的模型导出为 ONNX 格式，从而摆脱对 Python 运行环境的依赖，二是可以完成推理性能的优化，支持图优化、算子融合等加速技术，即便在 CPU 环境下也能实现不错的运行速度。

3.1.2 后端框架选型

后台服务依靠 Flask 框架完成搭建，选用该框架的原因可以分为多个层面，一是拥有轻巧简约的运行特质，这款轻量化框架具备精简的核心功能，能够很好适配该系统的服务运行体量，二是具备极强的适配灵活度，框架不会限定专属的数据库与页面渲染工具，各类功能组件都能依照实际使用需求自由搭配选用，三是有着完备的配套资源体系，依托成熟底层组件搭建而成的它，还配备有数量充足且实用性较强的拓展工具，四是能够实现便捷的上线布设，简易的搭建流程可以被轻松完成，较低的硬件资源需求也能有效降低服务器的运行负担。

3.1.3 数据库选型

能够实现数据长久保存的工作由系统选用的 MySQL 关系型数据库来完成，做出此项选用安排有着多方面的实际缘由，一是具备成熟可靠的运行特质，这款开源数据库有着十分广泛的应用范围，各类实际运行场景都已经对它的稳定性能做出了充分验证，二是拥有完备的事务处理能力，完整的事务机制能够被其提供，数据执行操作过程里的完整性与统一性也能由此得到保障，三是配备完善的数据调取语法，能够轻松完成各类信息检索与数据汇总整理的相关工作，四是可以和 Flask 开发体系做到顺畅融合，借助对应的程序拓展工具便能轻松完成各类数据读写相关操作。

3.1.4 前端技术选型

整套前端界面依靠原生的网页编写语言搭建而成，全程没有借助各类主流开发框架，做出这样的选型有着多重缘由，一是该系统的核心作用集中在画面展示与识别结果呈现之上，整体交互流程较为简易，各类体积偏大的前端架构也就没有引入的必要，二是原生编写模式能够减少配套编译工具的使用，得以把系统上线部署的繁琐程度降到更低，

三是网页自带的各类实用接口能够被充分调用，文件上传与提前预览的操作可借助专属接口完成，画面绘制工作能够依靠绘图接口实现，新式布局样式还可轻松适配不同尺寸的显示设备。

3.1.5 ONNX Runtime 推理加速方案

微软开源的 ONNX Runtime 是高性能推理工具，能为系统深度学习模型部署推理提供关键支持，系统用它对三类模型做推理加速：一是去雾、去雨的 NAFNet 模型，导出 ONNX 格式后用该引擎加载，开启全图优化、设 4 线程调动多核性能，还通过会话缓存复用会话；二是低光照增强的 MBLLEN 模型，除原生推理路径外，新增该引擎推理方案，能显著提升速度并支持 4 线程并行；三是同用于低光照增强的 Zero-DCE 模型，也用该引擎提速，减少对特定深度学习框架的依赖。

3.1.6 技术选型对比总结

下表 3-1 总结了系统关键技术的选型情况及选择依据：

表 3-1 系统关键技术选型

技术维度	选型方案	备选方案	选型理由
深度学习训练框架	PyTorch	TensorFlow	生态活跃，调试便捷，学术主流
模型推理引擎	ONNX Runtime	TensorRT, OpenVINO	跨框架兼容，CPU 优化好，部署简便
后端框架	Flask	Django, FastAPI	轻量灵活，部署简单，扩展性强
数据库	MySQL	SQLite, PostgreSQL	成熟稳定，事务支持，查询高效
前端	HTML5+CSS3+JS	React, Vue	轻量无依赖，满足系统需求
图像处理库	OpenCV	Pillow, scikit-image	功能完善，性能优异，生态成熟
开发语言	Python 3.12	—	算法生态丰富，开发效率高

3.2 图像增强算法

图像增强属于这套系统里十分关键的处理环节，能够对恶劣天气中画质受损的路面图像完成修复优化，把画面整体品质做到有效提升，也能为后续开展的车辆识别工作提供优质的原始图像数据，该系统的图像增强模块运用多类方法结合的设计思路，把深度智能优化模型和传统画面处理算法进行整合运用，可全面适配雾天、雨天以及光线不足这三类常见的不良拍摄环境。

3.2.1 图像增强模块整体架构

图像增强服务以 ImageEnhancer 作为核心操作类，此类采用单例形式完成设计，能够让全局范围内只保留唯一的增强运行实例，相关实例创建规则依靠重写内置方法与状态变量进行管控，重复新建模型所造成的内存资源损耗也能由此得到有效规避，增强模块的内部架构依照策略模式完成搭建，统一的图像处理接口能够向外提供各类调用服务，该接口可以接入图像地址、保存路径以及优化类别三类参数，还能依照传入的类别参数自动分配至对应的处理逻辑当中，这样的架构形式在增添全新优化算法时，仅需补充对应的映射关联即可，调用端程序无需做出改动，良好的拓展性能也由此得到体现，各类增强组件还被系统以延迟加载的方式统筹管理，相关组件被设置为属性形式，只有在首次被调取使用时才会完成初始化流程，系统启动阶段因批量加载模型而产生的启动卡顿问题能够被有效化解，每一类增强组件内部都增设了模型文件核验机制，缺失相关文件时会给出清晰提示并自动切换至备用处理方式，系统整体运行的稳定程度也得到了有力保障^[10]。如下图 3-1 所示：

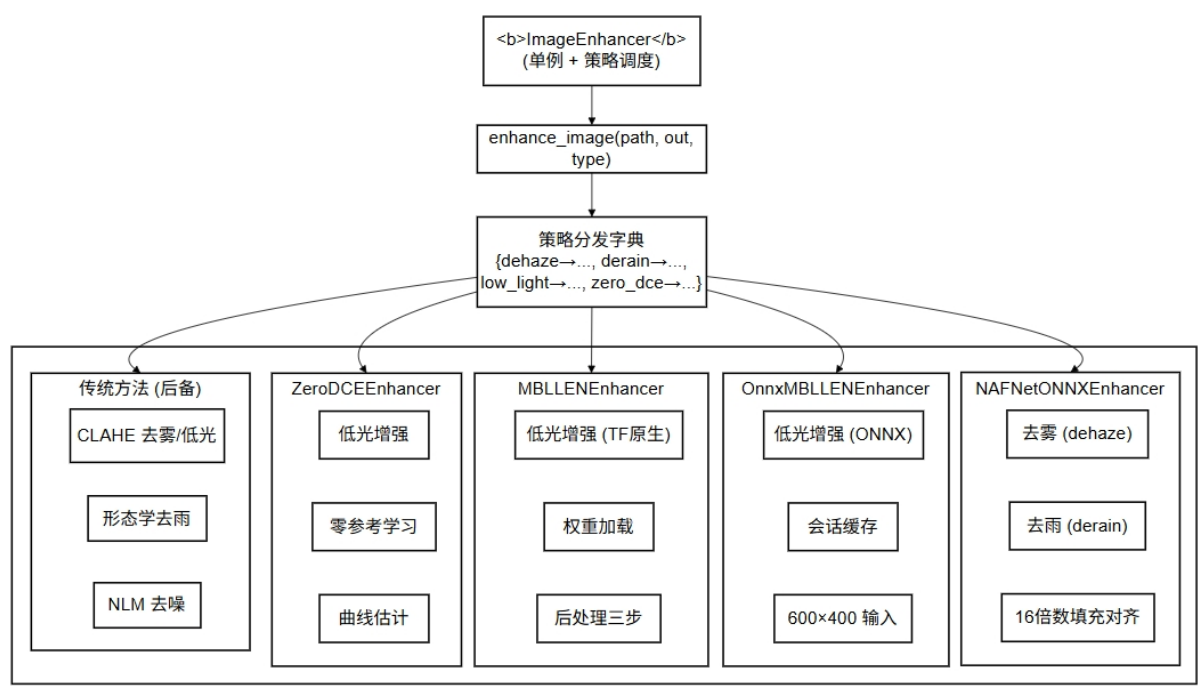


图 3-1 增强模型结构图

3.2.2 基于 NAFNet 的去雾与去雨增强

NAFNet (Nonlinear Activation Free Network) 是一种简洁而高效的图像恢复网络架构。与传统的图像恢复网络不同, NAFNet 摒弃了复杂的非线性激活函数 (如 ReLU、Sigmoid 等), 转而采用简单的元素级门控机制, 在保持强大表示能力的同时大幅简化了网络结构。这种设计使得 NAFNet 在图像去雾和去雨任务中均能取得优异的性能表现。在系统的实现中, NAFNet 模型被导出为 ONNX 格式, 并通过 NAFNetONNXEnhancer 类进行封装。该类实现了完整的推理管线, 包含预处理、模型推理和后处理三个环节。

预处理阶段的核心代码实现了图像尺寸对齐和数据格式转换。这段代码的核心设计思路可以整理为, 先把输入图像从 OpenCV 常用的 BGR 颜色格式转换为 RGB 格式, 这是由于深度学习模型大多使用 RGB 格式完成训练, 再把像素数值从 0 到 255 的整数范围调整为 0.0 到 1.0 的浮点数范围, 让数据格式与模型训练时的标准保持一致, 随后将图像数据格式从高度、宽度、通道的排列方式转换为通道、高度、宽度的排列方式, 并增加一个批次维度, 最终形成标准的张量数据, 其中最关键的设计是加入了 16 的倍数填充对齐功能, 因为 NAFNet 模型会进行多次下采样处理, 要求输入图像的尺寸必须是 16 的倍数, 否则会出现输出图像和输入图像大小不一致的问题, 系统选用的是反射填充的方式来完成这项处理。代码如下:

```
def _preprocess(self, img):
    orig_h, orig_w = img.shape[:2]
    pad_h = (16 - orig_h % 16) % 16
    pad_w = (16 - orig_w % 16) % 16
    if pad_h > 0 or pad_w > 0:
        img = cv2.copyMakeBorder(img, 0, pad_h, 0, pad_w, cv2.BORDER_REFLECT)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_rgb = img_rgb.astype("float32") / 255.0
    img_chw = img_rgb.transpose(2, 0, 1)
    img_input = np.expand_dims(img_chw, axis=0)
    return img_input, orig_h, orig_w, pad_h, pad_w
```

后处理阶段将模型的输出张量转换为图像格式。后处理代码会把模型输出的数据从 CHW 格式还原为 HWC 格式, 通过数值约束操作让像素值稳定在 0 到 1 的有效区间内, 再把数值恢复到 0 至 255 的整数范围并转换回 BGR 色彩空间, 同时按照预处理记录的原始尺寸去掉多余的填充区域, 让图像恢复到原本的大小, 这项精准的尺寸对齐策略能让增强前后的图像尺寸保持完全相同, 对后续开展的车辆检测任务提供了重要保障。代码如下:

```
def _postprocess(self, output, orig_h, orig_w, pad_h, pad_w):
```

```

enhanced = output[0].transpose(1, 2, 0)
enhanced = np.clip(enhanced, 0, 1)
enhanced = (enhanced * 255).astype("uint8")
enhanced = cv2.cvtColor(enhanced, cv2.COLOR_RGB2BGR)
if pad_h > 0 or pad_w > 0:
    enhanced = enhanced[:orig_h, :orig_w]
if enhanced.shape[:2] != (orig_h, orig_w):
    enhanced = cv2.resize(enhanced, (orig_w, orig_h))
return enhanced

```

NAFNetONNXEnhancer 在进行 ONNX Runtime 推理优化的过程中,还采用了会话缓存机制来提升运行效率会话缓存机制会通过类变量_session_cache,把创建好的推理会话保存起来,当多个请求同时到达时,只有第一个请求会创建会话,后续的请求直接复用缓存里的会话,这样就避免了重复解析模型图,还能减少分配内存的开销,进而提升系统的并发处理能力,另外通过设置 intra_op_num_threads=4 来进行多线程并行计算,也充分把 CPU 的多核性能利用了起来。会话缓存代码如下:

```

class NAFNetONNXEnhancer:
    _session_cache = {}

    def __init__(self, model_path):
        if model_path in NAFNetONNXEnhancer._session_cache:
            self.session = NAFNetONNXEnhancer._session_cache[model_path]
        else:
            session_options = ort.SessionOptions()
            session_options.graph_optimization_level = \
                ort.GraphOptimizationLevel.ORT_ENABLE_ALL
            session_options.intra_op_num_threads = 4
            self.session = ort.InferenceSession(
                model_path,
                sess_options=session_options,
                providers=["CPUExecutionProvider"],
            )
            NAFNetONNXEnhancer._session_cache[model_path] = self.session

```

3.2.3 基于 MBLLN 的低光照增强网络

MBLLN (Multi-Branch Low-Light Enhancement Network) 是专门针对低光照图像增强所设计的深度神经网络, 这一网络的核心思路是借助多分支结构从不同感受野中提取图像的特征, 再通过融合模块把提取到的各类特征整合到一起, 最终生成完成增强处理的图像, 在系统中 MBLLN 增强器提供了两种推理实现方式, 一是 MBLLNEnhancer 依托 TensorFlow 框架完成实现, 二是 OnnxMBLLNEnhancer 依托 ONNX Runtime 完成实现, MBLLN 网络结构的定义用到了 TensorFlow 的 Keras API, 核心部分为 EM (Enhancement Module) 模块。

该网络设计的创新之处是采用特征提取模块 (FEM) 先对输入进行浅层特征提取, 然后通过多组 EM 模块进行多尺度特征提取与融合。每个 EM 模块内部采用编码器-解码器的结构。通过卷积操作逐步扩大通道数以捕获高层语义信息, 再通过转置卷积逐步恢复空间分辨率。这种编码器-解码器结构使得网络能够同时利用底层细节信息和高层语义信息。EM 模块之间通过 Concatenate 操作进行密集连接, 将所有分支的输出在通道维度上进行拼接, 使融合层能够访问到各个尺度的特征信息。代码如下所示:

```
def build_mbllen():
    def EM(input, kernel_size, channel):
        conv_1=layers.Conv2D(channel,(3,3),activation="relu",
padding="same")(input)
        conv_2 = layers.Conv2D(channel, (kernel_size, kernel_size), activation="relu",
padding="valid")(conv_1)
        conv_3=layers.Conv2D(channel * 2, (kernel_size, kernel_size),
activation="relu", padding="valid")(conv_2)
        conv_4=layers.Conv2D(channel * 4, (kernel_size, kernel_size),
activation="relu", padding="valid")(conv_3)
        conv_5=layers.Conv2DTranspose(channel * 2, (kernel_size, kernel_size),
activation="relu", padding="valid")(conv_4)
        conv_6=layers.Conv2DTranspose(channel, (kernel_size, kernel_size),
activation="relu", padding="valid")(conv_5)
        res = layers.Conv2DTranspose(3, (kernel_size, kernel_size), activation="relu",
padding="valid")(conv_6)
        return res
    inputs = layers.Input(shape=(None, None, 3))
    FEM = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(inputs)
    EM_com = EM(FEM, 5, 8)
```

```

for j in range(3):
    for i in range(0, 3):
        FEM=layers.Conv2D(32, (3, 3), activation="relu", padding="same")(FEM)
        EM1 = EM(FEM, 5, 8)
        EM_com = layers.Concatenate(axis=3)([EM_com, EM1])
    outputs = layers.Conv2D(3, (1, 1), activation="relu", padding="same")(EM_com)
return Model(inputs, outputs)

```

在低光照增强的后处理方面，系统实现了 MBLLEN 官方推荐的后处理流程。该后处理流程有三个关键的步骤：第一步是线性放大。通过分析灰度直方图的高百分位值，来计算出合适的放大系数并对图像进行线性拉伸，提升图片的整体的亮度；第二步为百分位重缩放，减去低百分位的暗部偏移值并重新映射到[0, 1]区间，增强暗部区域的细节可见性；第三步就是 HSV 空间的饱和度伽马校正。通过对饱和度分量应用指数变换，在保持色调不变的前提下增强了图片的色彩鲜艳度。这个后处理流程是 MBLLEN 算法和其他低光照增强方法的的重要区别。代码如下所示：

```

def postprocess(self, output, lowpercent=5, highpercent=95, maxrange=0.8,
hsvgamma=0.8):
    fake_B = output[0, :, :, :3].copy()
    gray_fake_B = fake_B[:, :, 0] * 0.299 + fake_B[:, :, 1] * 0.587 + fake_B[:, :, 2] *
0.114
    percent_max = np.sum(gray_fake_B >= maxrange) / np.sum(gray_fake_B <= 1.0)
    max_value = np.percentile(gray_fake_B[:, :], highpercent)
    if percent_max < (100 - highpercent) / 100.0:
        scale = maxrange / max_value
        fake_B = fake_B * scale
        fake_B = np.minimum(fake_B, 1.0)
    gray_fake_B = fake_B[:, :, 0] * 0.299 + fake_B[:, :, 1] * 0.587 + fake_B[:, :, 2] *
0.114
    sub_value = np.percentile(gray_fake_B[:, :], lowpercent)
    if sub_value < 0.5:
        fake_B = (fake_B - sub_value) * (1.0 / (1 - sub_value))
    imgHSV = cv2.cvtColor(fake_B, cv2.COLOR_RGB2HSV)
    H, S, V = cv2.split(imgHSV)
    S = np.power(S, hsvgamma)
    imgHSV = cv2.merge([H, S, V])

```

```

fake_B = cv2.cvtColor(imgHSV, cv2.COLOR_HSV2RGB)
fake_B = np.minimum(fake_B, 1.0)
return fake_B

```

3.2.4 基于 CLAHE 和形态学操作的传统增强方法

深度学习增强模型虽然在效果上有优势，但在模型文件缺失或者加载失败的时候，系统需要具备降级能力。为此，系统还实现了基于传统图像处理方法的增强方案作为后备。

CLAHE (Contrast Limited Adaptive Histogram Equalization) 去雾增强采用限制对比度自适应直方图均衡化算法。该算法的主要思想是将图像划分为若干小块，在每个小块内执行直方图均衡化，同时还要设置对比度限制阈值以防止噪声被过度的放大。在实现中，系统对每个颜色通道分别进行了 CLAHE 处理^[4]，代码如下所示：

```

def dehaze(self, image_path, output_path):
    img = cv2.imread(image_path)
    if img is None:
        return None
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = img.copy()
    for i in range(3):
        enhanced[:, :, i] = clahe.apply(img[:, :, i])
    cv2.imwrite(output_path, enhanced)
    return output_path

```

这种逐通道 CLAHE 处理的优势在于：（1）计算速度快，无需加载深度学习模型；（2）参数设置简单，只需调整 clipLimit 和 tileGridSize 这两参数；（3）对均匀雾霾场景有显著的改善效果。系统经过实验调优，将 clipLimit 设置为 2.0，tileGridSize 设置为 (8, 8)，在该配置下取得了增强效果和计算效率的良好平衡。

低光照增强方面，系统采用 LAB 色彩空间下的 CLAHE 增强策略。这个方法的实现思路就是将图像从 BGR 色彩空间转换到 LAB 色彩空间，其中 L 通道是亮度信息，A 和 B 通道是颜色信息。只在 L 通道上使用 CLAHE 增强，可以有效地提升图像亮度和对比度，同时能避免对颜色信息造成干扰。clipLimit 设置到 3.0，略高于去雾场景，来应对低光照图像更严重的退化程度。代码如下所示：

```

def enhance_low_light(self, image_path, output_path):
    img = cv2.imread(image_path)
    if img is None:

```

```

        return None
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l_channel, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    l_channel = clahe.apply(l_channel)
    enhanced = cv2.merge([l_channel, a, b])
    enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)
    cv2.imwrite(output_path, enhanced)
    return output_path

```

雨天增强采用形态学开运算结合快速非局部均值去噪的方法。形态学开运算（先腐蚀后膨胀）是可以有效的去除图像中的细小亮色噪点，雨滴在图像中表现为细小的亮色条纹，因此开运算可以起到初步的去雨效果。后续的快速非局部均值去噪（fastNlMeansDenoisingColored）利用图像中的自相似性，在保留边缘细节的同时进一步平滑掉剩余的雨痕噪声。代码如下所示：

```

def derain(self, image_path, output_path):
    img = cv2.imread(image_path)
    if img is None:
        return None
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    opened = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
    denoised = cv2.fastNlMeansDenoisingColored(opened, None, 10, 10, 7, 21)
    cv2.imwrite(output_path, denoised)
    return output_path

```

3.3 自动场景识别算法

自动场景识别是系统实现智能化和自适应增强的关键技术。该算法分析输入图像的多维质量指标，自动的判断当前的天气条件类型，选择出最优的增强策略。

3.3.1 算法设计与原理

自动场景识别算法封装在 AutoEnhancer 类里，用了单例模式设计，算法的运行逻辑都在 detect_from_frame 方法中，它能从输入的 BGR 图像里，提取出四个维度的质量特征，一是亮度特征，把彩色图像转成灰度图后，算出所有像素值的均值，低光照图像的亮度均值通常比较低；二是对比度特征，就是灰度图像像素值的标准差，雾天图像的对比度会明显降低；三是饱和度特征，把图像转到 HSV 色彩空间，算出饱和度通道的均值，雨天和雾天图像的饱和度都会下降；四是清晰度特征，对灰度图用拉普拉斯算子，

算出响应的方差，雾天图像的清晰度数值会偏低

基于上述四个特征，算法采用级联决策规则进行场景分类，其判断流程如下图 3-2 所示：

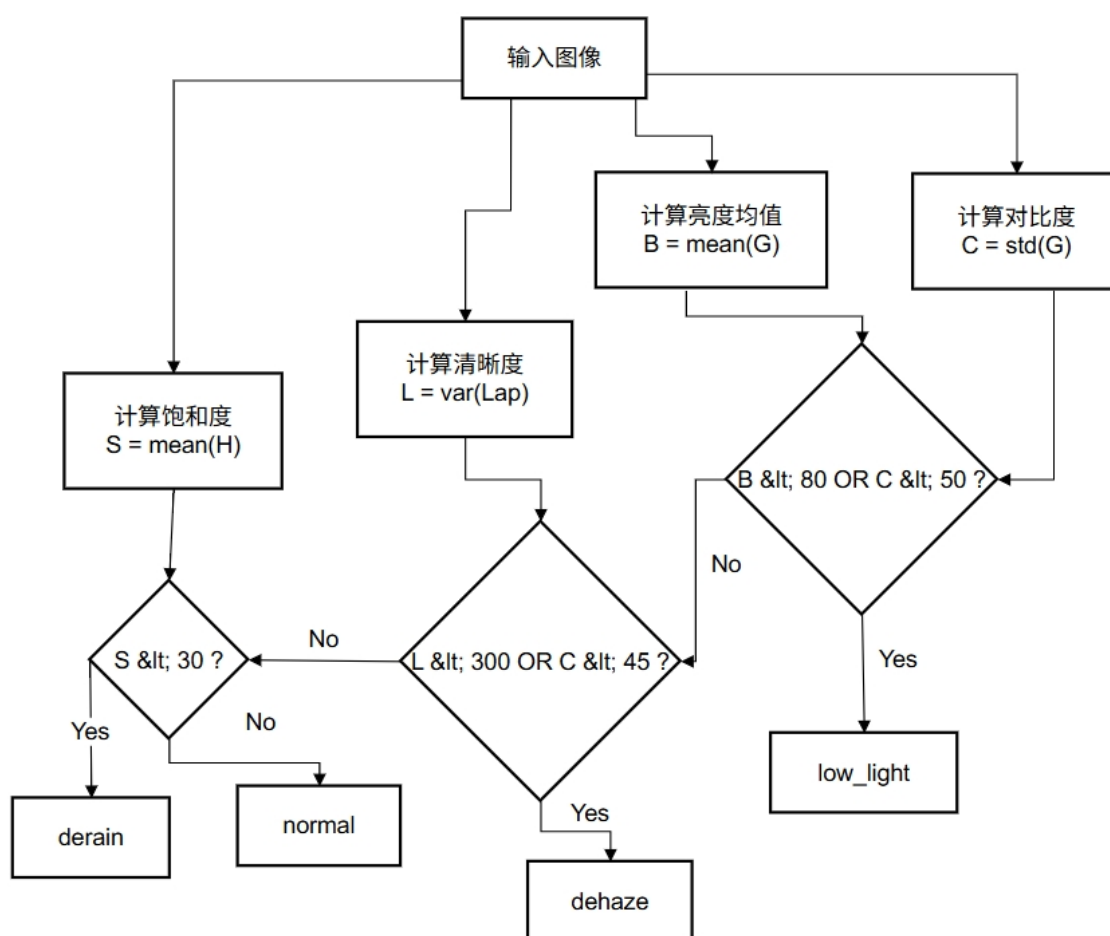


图 3-2 场景识别决策路径图

首先判断是否为低光照场景（亮度低于 80 或对比度低于 50），然后判断是否为雾天场景（清晰度低于 300 或对比度低于 45），再判断是否为雨天场景（饱和度低于 30），最后默认为正常天气。四种天气类型的典型图像质量指标数值范围如下表 3-2 所示：

表 3-2 各天气类型图像质量指标典型值

天气类型	亮度 (0-255)	对比度 (std)	饱和度 (0-255)	清晰度 (var)
正常天气	120-180	55-90	45-80	>500
雾天	80-150	20-45	20-40	<300
雨天	70-130	35-60	10-30	200-600
低光照	20-80	15-50	25-55	100-400
决策阈值	B < 80 (低光)	C < 45 (低光)	S < 30 (雨天)	L < 300 (雾天)

从表 3-1 可看出，低光照图像的亮度和对比度指标显著低于正常天气，是区分度最高的特征；雾天图像将低清晰度和低对比度当作典型的特征；雨天图像则以低饱和度为最显著标志。决策阈值的设定遵循优先级从高到低的原则——低光照优先判断（因其亮度特征最易区分），雾天次之，雨天最后，没有命中任何判据则归为正常天气。

3.3.2 阈值确定与优化

场景识别算法的核心是一组预设的决策阈值，这些阈值定义在 DEFAULT_THRESHOLDS 字典中：

```
DEFAULT_THRESHOLDS = {  
    "low_light_brightness": 80,  
    "low_light_contrast": 50,  
    "dehaze_sharpness": 300,  
    "dehaze_contrast": 45,  
    "derain_saturation": 30,  
}
```

这些阈值是通过在大量带有标注的恶劣天气图像上进行统计分析确定的。具体而言，从数据集中分别选取雾天、雨天、低光照和正常天气的图像样本，计算出各样本的四维质量指标，绘制各指标在不同天气类型下的分布曲线，然后取分类精度最高的分界点作为阈值。

该算法的创新点有这么几个方面：（1）该算法采用基于规则的轻量级决策方案，避免了训练深度学习分类器所需的标注成本和计算开销。（2）本算法四个维度的特征提取计算复杂度极低，场景识别过程耗时可忽略不计。（3）算法的决策规则采用级联结构，优先级从高到低排列，有效处理了多天气条件重叠的边界情况。（4）算法的阈值参数化设计，可以通过构造函数传入自定义阈值，便于针对不同应用场景进行调优。

3.4 基于 YOLOv8 的车辆检测算法

3.4.1 YOLOv8 模型概述

YOLOv8 是 Ultralytics 公司在 2023 年推出的模型，也是 YOLO 系列算法的最新更新版本，它在沿用 YOLOv5 优秀工程实践的基础上，还做了好几处架构上的改进，一是采用了 Anchor-Free 检测头，把预定义锚点框的超参数调优过程给省去了^[16]；二是设计了解耦头结构，会把分类和回归这两个任务，分配到各自独立的卷积分支上，解决了多任务学习时的竞争问题；三是加入了 C2f 模块，用来尝试替换原来的 C3 模块，这样做也增强了梯度流动和特征复用的效果；四是支持 n/s/m/l/x 等多种模型规格，能满足不同计算资源条件下的部署需求^[11]。

3.4.2 车辆检测器的实现

系统将 YOLOv8 模型封装在 `VehicleDetector` 类中。该类实现完整的检测管线，包括模型加载、目标检测和结果统计。

模型加载采用缓存机制，避免在多帧连续检测场景中重复创建模型实例。这个实现的设计要点主要有三种，一是能自动搜索模型文件的路径，会优先选用 ONNX 格式，要是没有再用 PyTorch 格式；二是通过类变量 `_model_cache`，实现了多实例之间的模型共享，能保证同一个模型权重，在内存里只加载一次，不会重复加载；三是模型规格可以进行配置，默认会用 YOLOv8x 规格，这样能让使用者获得最好的检测精度。代码如下所示：

```
class VehicleDetector:
    _model_cache = {}

    @staticmethod
    def get_model_path(model_name="yolov8x"):
        base = os.path.join(WEIGHTS_DIR, "detection")
        onnx_path = os.path.join(base, f"{model_name}.onnx")
        pt_path = os.path.join(base, f"{model_name}.pt")
        if os.path.exists(onnx_path):
            return onnx_path
        if os.path.exists(pt_path):
            return pt_path
        return f"{model_name}.pt"

    def __init__(self, weights_path=None):
        if weights_path is None:
            weights_path = self.get_model_path("yolov8x")
        if weights_path in VehicleDetector._model_cache:
            self.model = VehicleDetector._model_cache[weights_path]
        else:
            self.model = YOLO(weights_path)
            VehicleDetector._model_cache[weights_path] = self.model
```

车辆检测的核心逻辑在 `detect_vehicles` 方法中实现。这套处理方式依靠文件路径完成数据的传入与导出，有着简洁明朗的整体设计，一是提前设定好了对应的车辆种类对应规则，COCO 数据集中的编号 1 能够匹配到自行车，编号 2 能够匹配到私家轿车，

编号 3 能够匹配到摩托车, 编号 5 能够匹配到公共巴士, 编号 7 能够匹配到货运卡车, 二是完成目标检测结果的绘制工作时, 类别名称与可信数值都会被标注在目标框的左上位置, 直观呈现出来的内容也能够方便使用者判断出最终检测结果的可靠程度。

```
def detect_vehicles(self, image_path, output_path):
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f'无法读取图像: {image_path}')
    results = self.model(img)
    counts = {"car": 0, "truck": 0, "bus": 0, "motorcycle": 0, "bicycle": 0}
    for result in results:
        for box in result.bboxes:
            cls = int(box.cls[0])
            if cls == 2: counts["car"] += 1
            elif cls == 7: counts["truck"] += 1
            elif cls == 5: counts["bus"] += 1
            elif cls == 3: counts["motorcycle"] += 1
            elif cls == 1: counts["bicycle"] += 1
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(img, f'{YOLO_CLASSES[cls]} {box.conf[0]:.2f}',
                        (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
255, 0), 2)
    cv2.imwrite(output_path, img)
    return counts
```

3.4.3 模型规格与性能权衡

YOLOv8 支持 n/s/x 三种规格, 模型参数量和计算复杂度依次递增。各规格的关键参数对比如下表 3-3 所示:

表 3-3 YOLOv8 各规格参数对比

规格	参数量 (M)	mAP50 (COCO)	推理速度 (ms/img)	模型大小 (MB)
YOLOv8n	3.2	0.902	15	6
YOLOv8s	11.2	0.925	28	22
YOLOv8x	68.2	0.951	105	132

本系统默认采用 YOLOv8x 规格以获得最高检测精度。对于计算资源受限的部署场景，系统支持通过 API 参数切换模型规格，在检测精度和处理速度之间取得平衡。

检测评估中，IoU 是判断预测框与真实框是否匹配的基础指标，如公式(3-1)如所示：

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|} \quad (3-1)$$

其中 B_{pred} 是预测边界框， B_{gt} 是真实边界框。IoU 值越大表明这两个框的重合度越高。系统在检测评估时， $\text{IoU} \geq 0.5$ 被视为正确检测。

3.5 ONNX Runtime 推理优化

3.5.1 多模型管理方案

系统在 ImageEnhancer 类中统一管理多个增强模型实例，通过惰性加载属性确保各增强器按需初始化。该方案的设计好处是：单例模式保证了全局只有一个 ImageEnhancer 实例；惰性加载保证了不会在系统启动的时候加载所有的模型，节省了内存资源；模型文件存在性检查使得在模型缺失时能够优雅降级到传统方法。代码如下所示：

```
class ImageEnhancer:
    __instance = None
    __initialized = False

    def __new__(cls):
        if cls.__instance is None:
            cls.__instance = super().__new__(cls)
        return cls.__instance

    def __init__(self):
        if ImageEnhancer.__initialized:
            return
        ImageEnhancer.__initialized = True
```

```

self._onnx_nafnet_dehaze = None
self._onnx_nafnet_derain = None
self._mbllen = None

@property
def onnx_nafnet_dehaze(self):
    if self._onnx_nafnet_dehaze is None:
        dehaze_path = os.path.join(WEIGHTS_DIR, "enhancement",
"nafnet_dehaze.onnx")
        if os.path.exists(dehaze_path):
            self._onnx_nafnet_dehaze = NAFNetONNXEnhancer(dehaze_path)
    return self._onnx_nafnet_dehaze

```

3.5.2 Zero-DCE 低光照增强的 ONNX 实现

Zero-DCE（Zero-Reference Deep Curve Estimation）不需要配对训练数据。系统通过 ZeroDCEEnhancer 类将其集成，并采用 ONNX Runtime 来进行推理加速。Zero-DCE 的推理管线与 NAFNet 类似，但处理逻辑略有不同：Zero-DCE 的输入不需要 16 的倍数填充对齐，但其输出尺寸可能与输入尺寸不一致，因此需要增加尺寸检查与调整的步骤。这种差异反映了不同模型架构对输入输出约束的差异，系统通过各增强器独立实现预处理和后处理逻辑来灵活适配。代码如下所示：

```

class ZeroDCEEnhancer:
    def __init__(self, model_path):
        import onnxruntime as ort
        self.session = ort.InferenceSession(
            model_path, providers=["CPUExecutionProvider"])
        self.input_name = self.session.get_inputs()[0].name

    def enhance_frame(self, img):
        orig_h, orig_w = img.shape[:2]
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype("float32") /
255.0

        img_chw = img_rgb.transpose(2, 0, 1)
        img_input = np.expand_dims(img_chw, axis=0)
        output = self.session.run(None, {self.input_name: img_input})[0]

```

```

enhanced = output[0].transpose(1, 2, 0)
enhanced = np.clip(enhanced, 0, 1)
enhanced = (enhanced * 255).astype("uint8")
enhanced = cv2.cvtColor(enhanced, cv2.COLOR_RGB2BGR)
if enhanced.shape[:2] != (orig_h, orig_w):
    enhanced = cv2.resize(enhanced, (orig_w, orig_h))
return enhanced

```

3.6 性能评测系统

3.6.1 评估指标体系

想要完成对不同天气环境下系统运行效果的综合评定，完整的性能评测组件已经被搭载到整套系统当中，这套评测体系划分出三个评定层面，一是检测效果相关指标，其中包含设定交并比数值为 0.5 时所得出的平均精度、交并比处于 0.5 至 0.95 区间内的平均精度、判定准确率、目标召回率以及各类车辆对应的平均精度，多项数值可以从不同层面完成对车辆识别精准程度的判定，二是画面质量相关指标，主要由峰值信噪比与结构相似性两项内容组成，依靠像素之间的误差数值能够把增强处理前后画面产生的差别体现出来，而结构相似性则可以结合明暗程度、色彩对比以及画面结构，完成对图像整体品质相近程度的判定，三是运行效率相关指标，把图像优化所用时长与目标识别所用时长都纳入统计范围，统一采用毫秒作为计量单位，以此完成对整体运转速率的衡量。

3.6.2 评测指标实现

mAP 的计算是评估检测性能的核心，系统在 metrics.py 中实现了完整的 mAP 计算流程。其数学定义为：对所有类别，计算在不同召回率水平上的平均精度，再对所有类别的 AP 取均值。给定预测框集合与真实框集合，首先根据 IoU 阈值区分真阳性（TP）与假阳性（FP），然后计算累积的 Precision 与 Recall：

Precision 与 Recall

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN} \quad (3-2)$$

Average Precision (AP)

$$\text{AP} = \sum_{k=1}^n (\text{Recall}_k - \text{Recall}_{k-1}) \times \text{Precision}_k \quad (3-3)$$

mAP 与 mAP50-95

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c \quad (3-4)$$

$$mAP_{50:95} = \frac{1}{10} \sum_{t \in \{0.5, 0.55, \dots, 0.95\}} mAP_t \quad (3-5)$$

其中 C 为类别数， mAP_{50} 对应 $IoU=0.5$ 的单阈值评估， mAP_{50-95} 则对 10 个 IoU 阈值（0.5~0.95，步长 0.05）的 mAP 取平均，对检测框的定位精度要求更为严格。

关键的 mAP 计算函数如下：

```
def calculate_map(gt_boxes, pred_boxes, iou_threshold=0.5):
    if not gt_boxes:
        if not pred_boxes:
            return 1.0
        return 0.0
    if not pred_boxes:
        return 0.0
    pred_boxes_sorted = sorted(pred_boxes, key=lambda x: x[4], reverse=True)
    tp = [0] * len(pred_boxes_sorted)
    fp = [0] * len(pred_boxes_sorted)
    used = [False] * len(gt_boxes)
    for i, pred in enumerate(pred_boxes_sorted):
        best_iou = 0
        best_idx = -1
        for j, gt in enumerate(gt_boxes):
            if used[j] or pred[5] != gt[5]:
                continue
            iou = calculate_iou(pred[:4], gt[:4])
            if iou > best_iou:
                best_iou = iou
                best_idx = j
        if best_iou >= iou_threshold and best_idx != -1:
            used[best_idx] = True
            tp[i] = 1
        else:
            fp[i] = 1
    tp_cumsum = np.cumsum(tp)
    fp_cumsum = np.cumsum(fp)
```

```

precision = tp_cumsum / (tp_cumsum + fp_cumsum + 1e-10)
recall = tp_cumsum / len(gt_boxes)
ap = 0
prev_recall = 0
for p, r in zip(precision, recall):
    if r > prev_recall:
        ap += p * (r - prev_recall)
        prev_recall = r
return ap

```

该实现严格遵循 PASCAL VOC 挑战赛的 mAP 计算标准。具体流程为：首先将所有预测框按置信度从高到低排序；然后遍历每个预测框，计算其与同类别真实框的最大 IoU，若超过阈值则视为真阳性（TP），否则为假阳性（FP）；接着计算累积的 Precision-Recall 曲线；最后通过插值法计算曲线下的面积作为 Average Precision（AP）。calculate_map_range 函数进一步计算 IoU 阈值从 0.5 到 0.95（步长 0.05）共 10 个阈值下的 AP 均值，得到 mAP50-95。

PSNR 和 SSIM 的数学定义如下。

PSNR（峰值信噪比）基于均方误差衡量两幅图像的像素级差异。

PSNR 公式如下所示：

$$MSE = \frac{1}{H \times W \times C} \sum_{x,y,c} (I_1(x,y,c) - I_2(x,y,c))^2 \quad (3-6)$$

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right), \quad MAX = 255 \quad (3-7)$$

SSIM（结构相似性）从亮度、对比度和结构三个维度综合评估图像质量：

SSIM 公式如下所示：

$$l(x,y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}, \quad c(x,y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}, \quad s(x,y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3} \quad (3-8)$$

$$SSIM(x,y) = [l(x,y)]^\alpha \cdot [c(x,y)]^\beta \cdot [s(x,y)]^\gamma \quad (3-9)$$

其中 μ 为局部均值， σ 为标准差， $C_1 = (0.01 \times 255)^2$ 、 $C_2 = (0.03 \times 255)^2$ 为稳定常数。

系统实现采用 $\alpha = \beta = \gamma = 1$ 的简化形式，以 11×11 高斯窗口（ $\sigma = 1.5$ ）逐像素滑动计算局部 SSIM 值，最终取全局均值作为整图的结构相似度得分。

PSNR 和 SSIM 的实现基于图像处理领域的标准计算方法：

```
def calculate_psnr_ssim(img1, img2):
    mse = np.mean((img1.astype(float) - img2.astype(float)) ** 2)
    if mse == 0:
        return 100.0, 1.0
    psnr = 10 * np.log10(255.0 ** 2 / mse)
    C1 = (0.01 * 255) ** 2
    C2 = (0.03 * 255) ** 2
    mu1 = cv2.GaussianBlur(img1_f, (11, 11), 1.5)
    mu2 = cv2.GaussianBlur(img2_f, (11, 11), 1.5)
    sigma1_sq = cv2.GaussianBlur(img1_f ** 2, (11, 11), 1.5) - mu1_sq
    sigma2_sq = cv2.GaussianBlur(img2_f ** 2, (11, 11), 1.5) - mu2_sq
    sigma12 = cv2.GaussianBlur(img1_f * img2_f, (11, 11), 1.5) - mu1_mu2
    ssim_map = ((2 * mu1_mu2 + C1) * (2 * sigma12 + C2)) /\
                ((mu1_sq + mu2_sq + C1) * (sigma1_sq + sigma2_sq + C2))
    ssim = float(ssim_map.mean())
    return round(psnr, 4), round(ssim, 4)
```

SSIM 的计算采用 11x11 的高斯窗口估计局部统计量,其中 C1 和 C2 是稳定性常数,防止分母接近零时出现除零错误。

3.6.3 评测流程设计

评测流程由 BenchmarkRunner 类管理,其自动化执行流程如下图 3-3 所示:

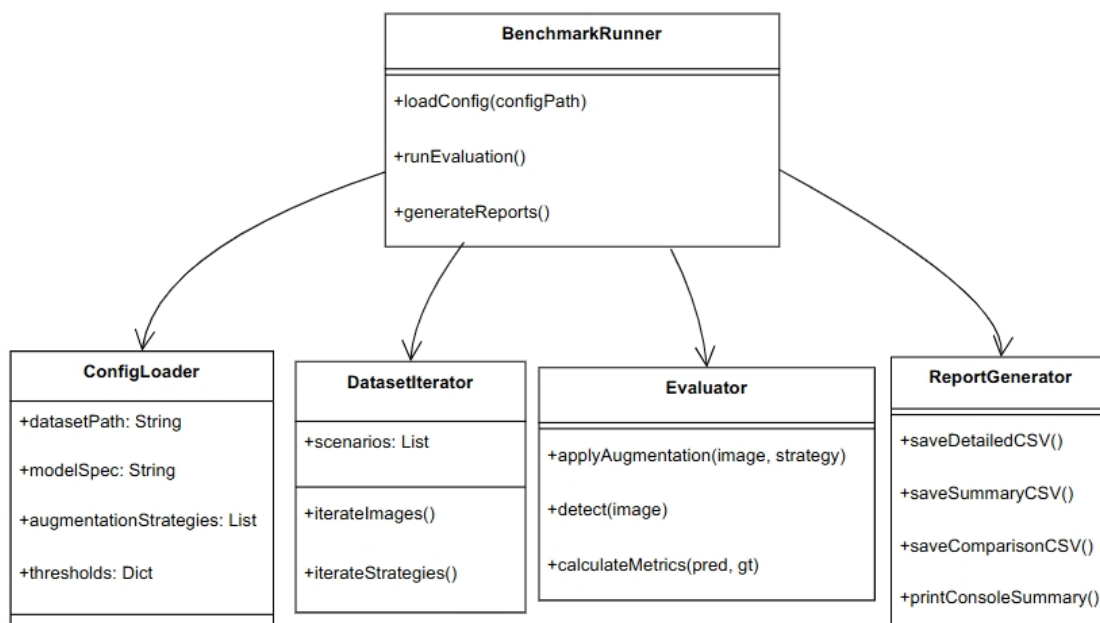


图 3-3 BenchmarkRunner 评测流程图

详细操作步骤如下，一是加载评测配置，从配置文件或者默认配置里，把数据集路径、模型规格、增强策略列表和检测参数都读取出来；二是遍历数据集，对正常、雾天、雨天、低光照这每个数据集里的所有图像，依次执行不同的增强策略和检测模型组合；三是对每张图像做完整评估，先判断要不要做增强处理，要是需要的话，就调用对应的增强方法来处理，之后再在增强后的图像上运行 YOLOv8 检测模型，算出检测性能指标和图像质量指标，再把所有评估结果都记录好；四是生成评测报告，通过 ReportGenerator 类，把评测结果输出成 CSV 文件，里面包含了详细结果、汇总统计和增强前后的对比分析；整个评测流程能完全自动化运行，还支持通过回调函数实时报告进度，方便在大规模数据集上做长时间的基准测试，这种可扩展的评测架构，也为算法的持续迭代优化提供了标准化的评估工具。

4. 系统设计与实现

4.1 系统总体架构

本系统采用前后端分离 B/S 架构，分为前端界面、后端服务、数据存储三层。前端通过接口与后端交互。后端处理业务与算法运算。数据层存储数据与文件，架构低耦合容易拓展。

用户在前端上传文件或实时取景，数据经接口传入 Flask 后端，后端完成图像增强、车辆检测等核心处理。结果存入存储端并以数据形式回传前端展示。

4.1.1 架构设计原则

本系统的架构设计遵循这样几个原则，一是分层架构，采用经典的三层架构模式，把系统分成前端表示层、后端业务逻辑层和数据持久化层，各层的职责都很明确，耦合度也比较低；二是前后端分离，前端和后端通过 RESTful API 来通信，前端负责用户交互和结果展示，后端负责处理业务逻辑、调用算法，这样能方便两者独立开发和部署；三是模块化设计，后端用模块化的方式来组织，把不同功能的处理逻辑都封装成独立的服务模块和路由模块，能提高代码的可维护性和可复用性；四是算法与业务解耦，把核心算法的实现和业务逻辑分离开来，算法模块会提供统一的调用接口，方便算法独立迭代和替换。

4.1.2 系统分层架构

系统的总体架构如图 4-1 所示，由上到下分为表示层、业务逻辑层和数据持久化层三个层次：

第一层：表示层（前端）

基于原生 HTML5+CSS3+JS 开发，通过 Fetch API 与后端接口异步通信。页面包含图像上传、结果展示、统计分析、历史记录四大功能区，可实时展示数据、查看详情与删除记录。

第二层：业务逻辑层（后端）

基于 Flask 框架，分为三个子层：

路由层：处理 HTTP 请求，含图像、视频、摄像头、历史记录四大路由模块

服务层：实现核心逻辑，提供图像增强、自动增强、车辆检测服务

算法层：依托 ONNX Runtime，调用 NAFNet、MBLLEN、YOLOv8 模型完成图像处理与检测。

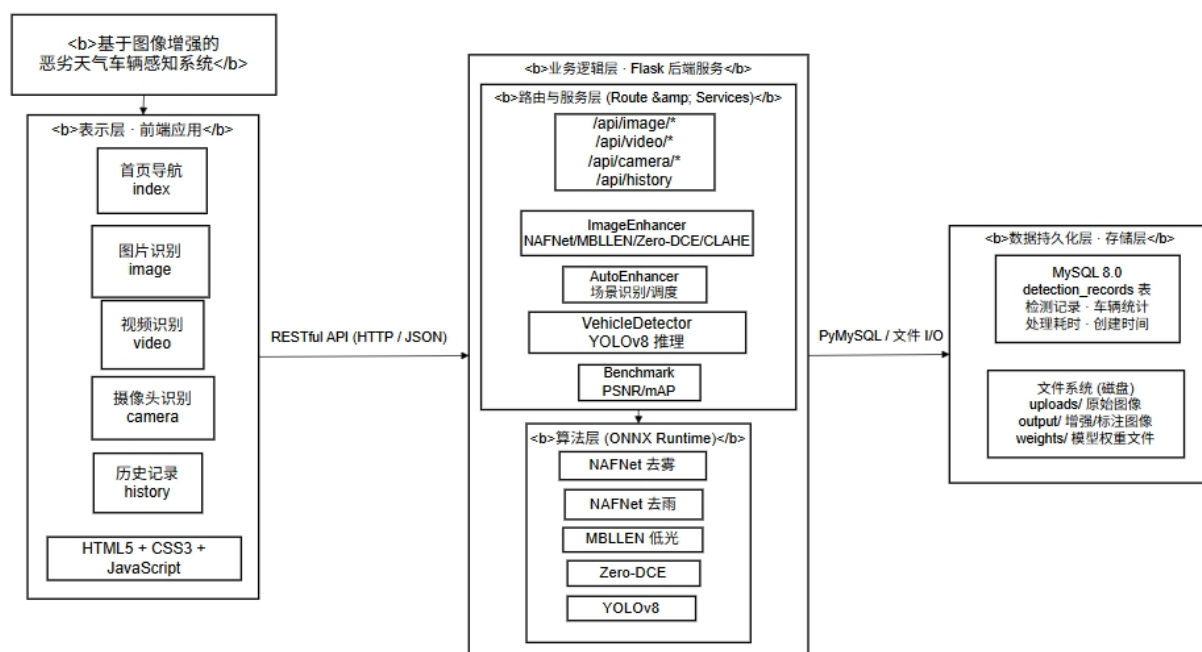


图 4-1 总体架构图

第三层：数据持久化层

数据持久化层采用 MySQL 关系型数据库和文件系统混合存储方案。结构化数据（检测记录、摄像头配置等）存储在 MySQL 数据库中；非结构化数据以文件形式存储在服务器的文件系统中，数据库记录中保存文件的路径引用。

4.1.3 系统模块划分

系统可从宏观上划分为前端、后端公共支撑、核心算法三个顶层包，各包之间的依赖关系如下图 4-2 所示：

从上图能看出来，这套系统是按分层架构来设计的，一共划分成了八个功能模块，一是 Web 前端模块，能提供用户操作界面，帮用户完成文件上传、图像预览、结果查看和历史记录管理；二是文件接收与管理模块，会接收用户上传的文件，做好格式校验和安全检测，还能统一管理文件的存储路径和访问地址；三是图像增强模块，这是系统的核心功能，集成了去雾、去雨、低光照提亮等相关算法，通过对应策略统一接口，能动态调用合适的增强工具；四是自动场景识别模块，会分析图像的亮度、对比度等相关指标，自动识别当前场景并匹配最适合的增强方案；五是车辆检测模块，依托 YOLOv8 完成目标识别，支持模型缓存和规格切换，还能输出车辆框坐标、类别、置信度以及数量统计；六是视频流处理模块，能支持视频解码和 RTSP 摄像头取流，逐帧做好增强和检测，再重新编码输出结果流；七是数据管理模块，负责连接 MySQL 数据库，做好数据的增删改查，管理好检测记录和摄像头配置；八是性能评测子系统，属于离线基准测试模块，能在标准数据集上算出 mAP、PSNR、SSIM 等相关指标，还能生成评测报告。

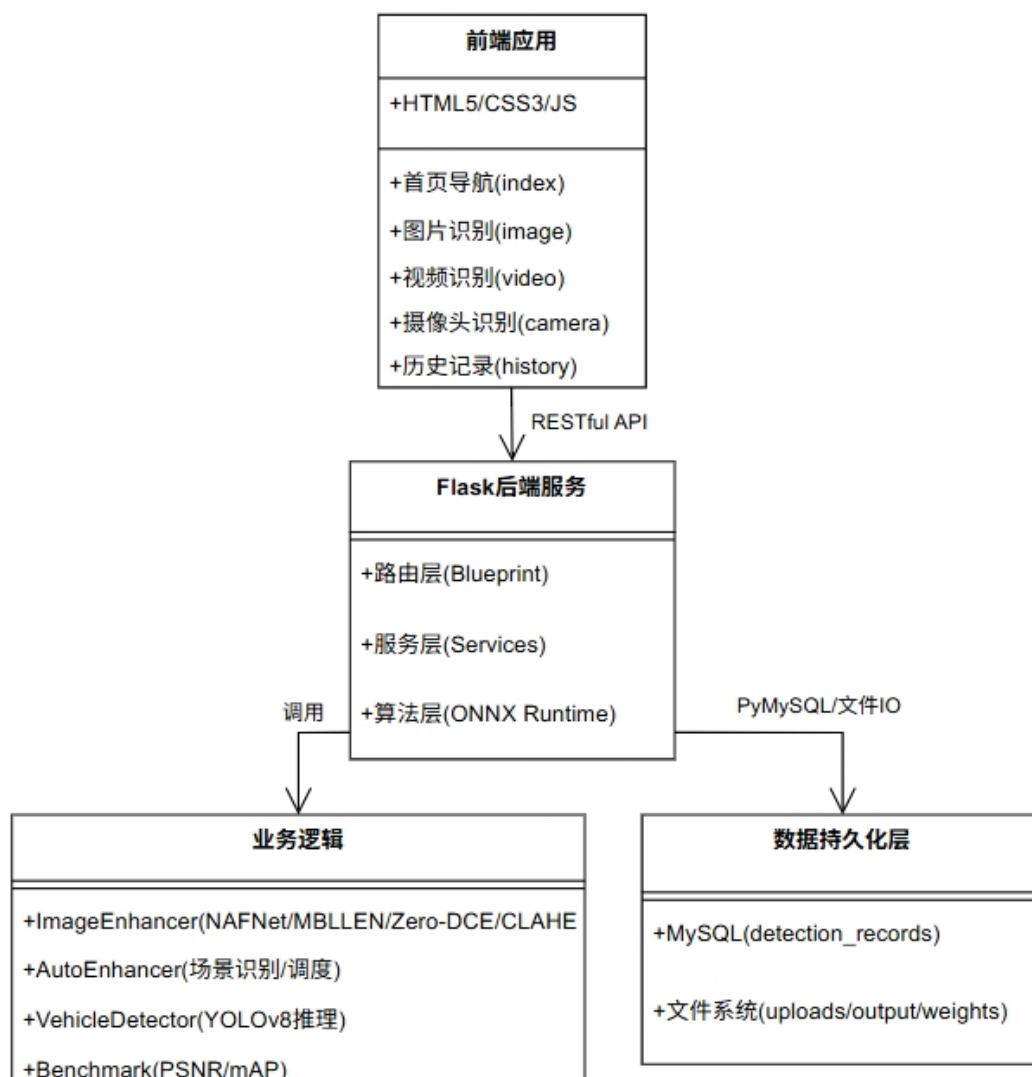


图 4-2 系统模块图

4.1.4 模块间相互关系

各模块之间的数据流和控制流关系如下所描述：

用户在 Web 前端上传图像，前端将图像文件的 multipart/form-data 格式发送至后端的图像上传 API。后端的路由层接收请求之后，将文件保存至文件系统，并调用服务层进行处理。服务层的自动场景识别模块首先对图像进行分析，判断天气条件类型。图像增强模块根据识别结果或用户指定的增强类型，调用相应的增强算法对图像进行恢复处理。增强后的图像传入车辆检测模块，让 YOLOv8 模型执行目标检测，输出检测结果。处理结果通过数据管理模块存储，会用 JSON 格式返回至前端展示。

在视频和摄像头模式下，处理流程类似，但增加了视频帧的解码和编码环节，并对每一帧重复执行增强和检测操作，最终合成结果视频返回给用户

4.1.5 数据处理流程设计

系统顶层数据流程图如下图 4-3 所示：

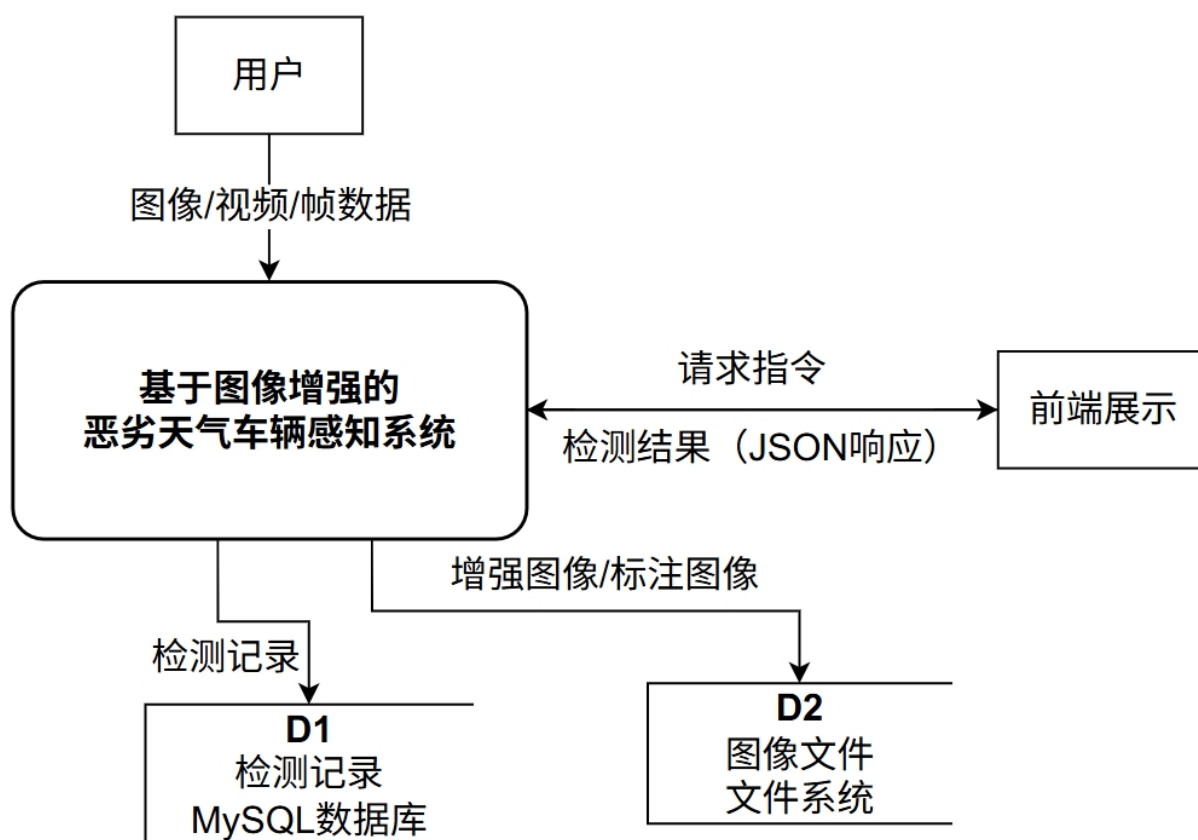


图 4-3 顶层数据流图

系统一层数据流程图如下图 4-4 所示：

完整的数据处理流程划分成五个执行环节，与一层数据流图中的五个加工一一对应。一是接收输入（加工 1），系统可通过网络请求接收用户上传的图像文件，也能借助视频解码工具读取本地视频文件与监控摄像头的实时画面帧数据；二是场景识别（加工 2），识别模块从输入图像中提取亮度、对比度、饱和度和清晰度四维质量参数，按照级联决策规则判定对应的天气类型，包括正常、雾天、雨天和低光照四种场景；三是图像增强（加工 3），系统根据场景识别结果自动调度适配的增强算法，雾天场景调用 NAFNet 去雾模块，雨天场景调用 NAFNet 去雨模块，低光照场景调用 MBLLEN 或 Zero-DCE 增强模块，正常场景则跳过增强步骤以节省运算资源；四是车辆检测（加工 4），增强处理后的图像被送入 YOLOv8 检测模型进行推理，模型输出包含检测框坐标、车辆类别和置信度的完整检测结果；五是结果封装（加工 5），该环节同时完成三项任务，将检测记录写入 MySQL 数据库的 detection_records 表，将增强图像与标注图像保存至文件系统，最后将所有结果统一封装为 JSON 响应通过网络返回前端页面完成展示。

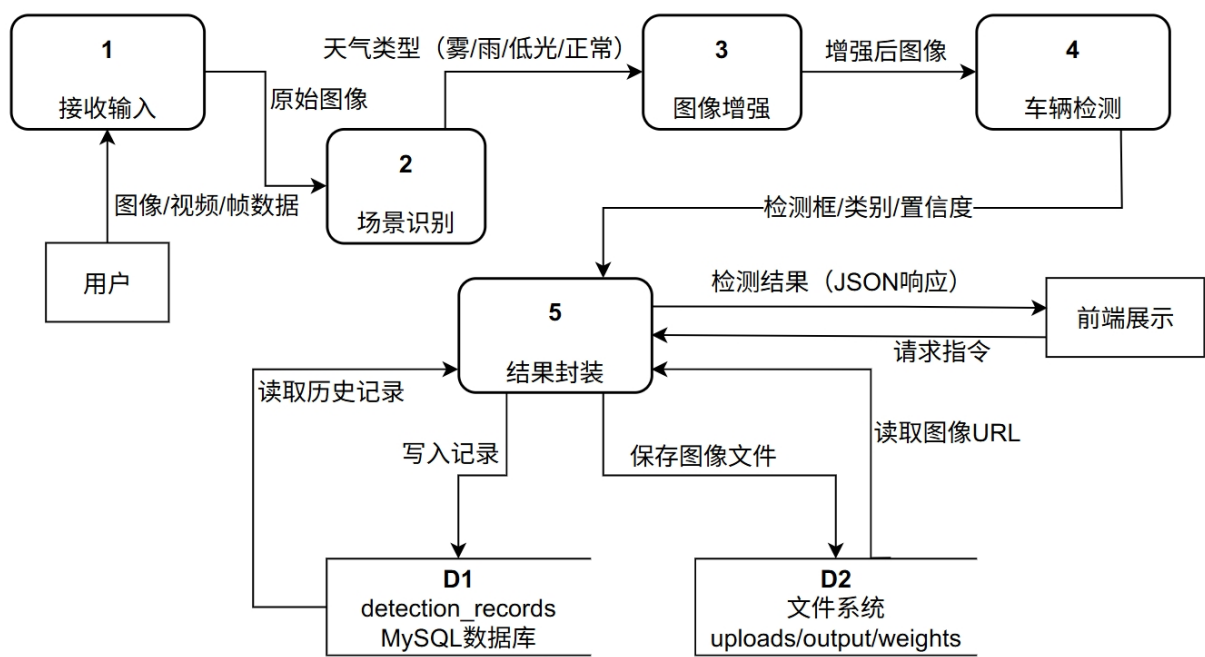


图 4-4 一层数据流图

4.2 前端设计与实现

4.2.1 页面结构设计

前端使用原生 HTML5+CSS3+JS 开发，无框架依赖，适配系统简单交互需求，部署便捷，充分调用 HTML5 与 CSS3 原生能力。

前端共六个页面：首页负责导航，图像 / 视频 / 摄像头页面实现对应检测功能，历史记录与测试结果页面展示数据。

页面采用统一深色主题，通过 CSS 变量构建样式体系，支持三色主题切换。用户设置本地持久化，视觉统一且无需额外加载样式文件

前端页面显示如下图 4-5 所示：



图 4-5 首页界面

4.2.2 图片识别模块

图片识别是前端核心模块，分为图像上传、图像增强、目标检测三流程。

用户选取图片后，前端读取预览并携带增强参数传至后端接口；后端处理后回传增强图片与耗时，支持去雾、去雨、暗光增强等多种优化模式。

随后前端提交图像与模型参数完成车辆检测。后端返回车型数量与标注图。页面展示五类车辆统计数据及总耗时。

页面采用三步流水线布局，分区展示原图、增强图与检测结果，搭配加载提示，操作直观清晰。

图片识别模块前端界面见下方图 4-6 所示：



图 4-6 图片识别界面

4.2.3 视频识别模块

视频识别界面和图片识别有着相近的运行思路，仅有处理载体换成了视频文件这一处不同，一是视频完成上传操作后，后台能够调用 OpenCV 工具完成视频逐帧的解码工作，还能对每一帧画面完成画质优化调整，再借助 VideoWriter 工具把优化完毕的画面重新整合编码，生成全新的成品视频，二就是目标检测环节依旧沿用逐帧运行的处理模式，而为了有效加快整体运行速率，还会启用间隔取帧的运行方式，每隔六帧画面仅做一次目标检测，其余画面直接用前一帧得出的检测数据，该种优化方式能维持整体检测流程的连贯状态，能缩减到原有六分之一的运算工作量，前端页面可依靠<video>标签呈现出原视频与经过优化处理后的成品视频，页面所展示出的统计数据为视频画面里出现过车辆最高数量，并非把全部画面里的车辆数量逐一相加得出的总计数值。

视频识别模块前端界面见下方图 4-7 所示：



图 4-7 视频识别界面

4.2.4 摄像头实时识别模块

摄像头实时识别是系统的一个功能，前端使用 WebRTC API 中的 navigator.mediaDevices.getUserMedia 接口获取摄像头视频流，在页面的<video>元素中实时显示。检测逻辑采用定时轮询机制，每次隔 1 秒从视频流中截取一帧进行处理。

采集到的视频帧通过 Canvas 绘制后编码为 JPEG 格式的 Base64 字符串，随增强模式和模型参数一并发送至后端的/api/camera/detect 接口。后端对帧进行增强和检测后返回检测统计数据。前端实时更新统计显示，实现接近实时的检测体验。

视频识别模块前端界面见下方图 4-8 所示：



图 4-8 摄像头识别界面

4.2.5 历史记录模块

历史记录页以表格展示检测数据，支持分页浏览、单删、批量删除与一键清空。分页接口每次加载十条记录，表格涵盖文件信息、车辆数量、效果图及时间等内容。

点击缩略图可弹窗全屏预览增强图与检测图，删除操作同步清除数据库数据与本地文件，保障数据统一。

历史记录识别模块前端界面见下方图 4-9 所示：



图 4-9 历史记录界面

4.2.6 测试结果分析页面

测试结果页面是系统评测数据的展示窗口，页面加载时自动请求 /api/benchmark/results 获取评测数据。页面按场景（雾天、雨天、低光照、正常）分类展示各增强方法在大规模基准测试中的性能表现，包括 mAP50、mAP50-95、PSNR、SSIM 等关键指标。数据以表格形式呈现，使用颜色编码增强可读性：mAP 提升以绿色标示，下降以红色标示；PSNR 超过 30dB 以绿色显示，低于 20dB 以红色显示。这种可视化的对比分析使用户能够直观地评估不同增强策略的效果差异。

测试结果模块前端界面见下方图 4-10 所示：



图 4-10 测试结果界面

4.3 后端路由设计与实现

后端采用 Flask 框架的 Blueprint（蓝图）机制组织路由，将不同功能的路由分散到独立的模块中。应用入口 `app.py` 负责创建 Flask 应用实例、加载配置、注册蓝图和启动服务。

四个 Blueprint 模块的划分及各自内部 API 端点的组织结构如下图 4-11 所示：

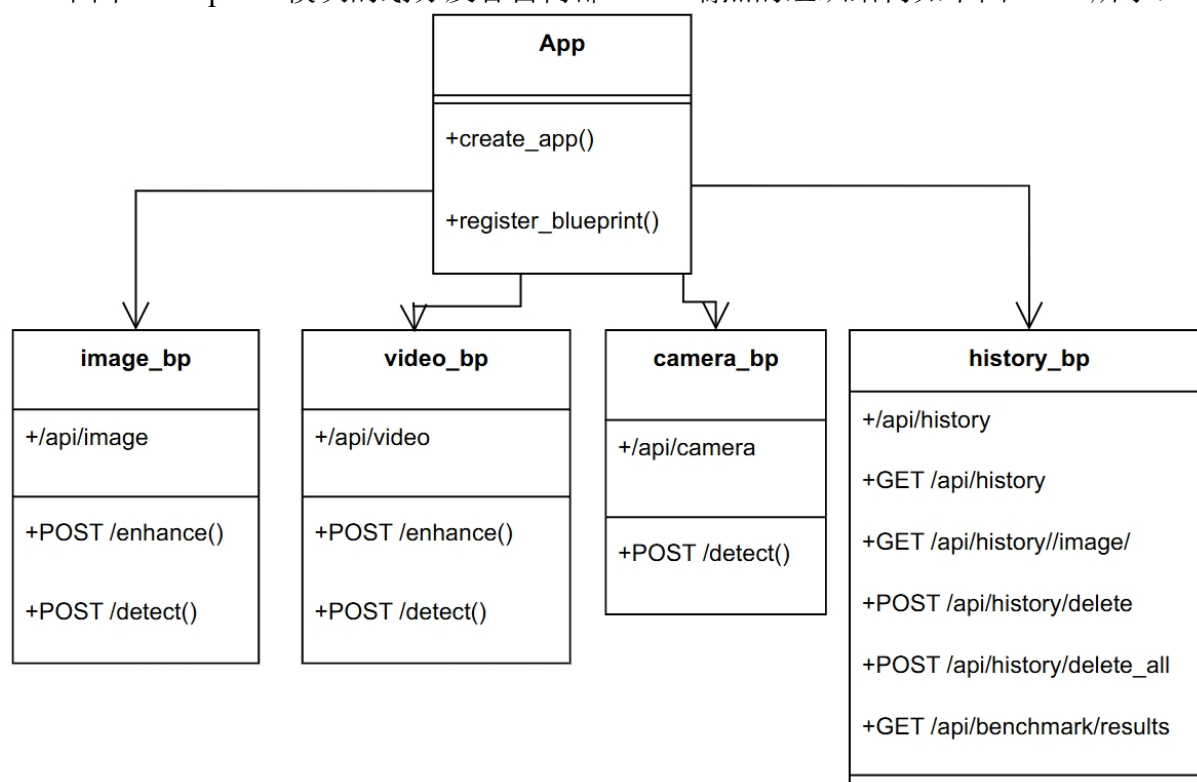


图 4-11 后端 Blueprint 路由组织结构图

该路由组织遵循“一个 Blueprint 一组 URL 前缀”的映射规则，每个 Blueprint 内部按 HTTP 方法继续细分路由，形成了清晰的树状结构。

以图片识别为例，一次完整的后端请求处理时序如下图 4-12 所示：

该路由组织遵循“一个 Blueprint 一组 URL 前缀”的映射规则，每个 Blueprint 内部按 HTTP 方法继续细分路由，形成了清晰的树状结构。

以图片识别为例，一次完整的后端请求处理时序如下图 4-12 所示：

各路由模块职责明确：`/api/image` 路由负责图像的上传、增强和检测；`/api/video` 路由负责视频文件的增强和检测；`/api/camera` 路由处理摄像头帧的实时检测；历史记录路由覆盖 CRUD 操作，同时提供 `benchmark` 评测数据的查询接口。

系统配置通过 `Config` 类统一管理，包含 MySQL 数据库连接参数、文件上传路径、允许的文件类型和最大文件大小限制。上传文件大小限制设置为 100MB，既满足了视频文件上传的需求，又防止了恶意的大文件上传攻击。

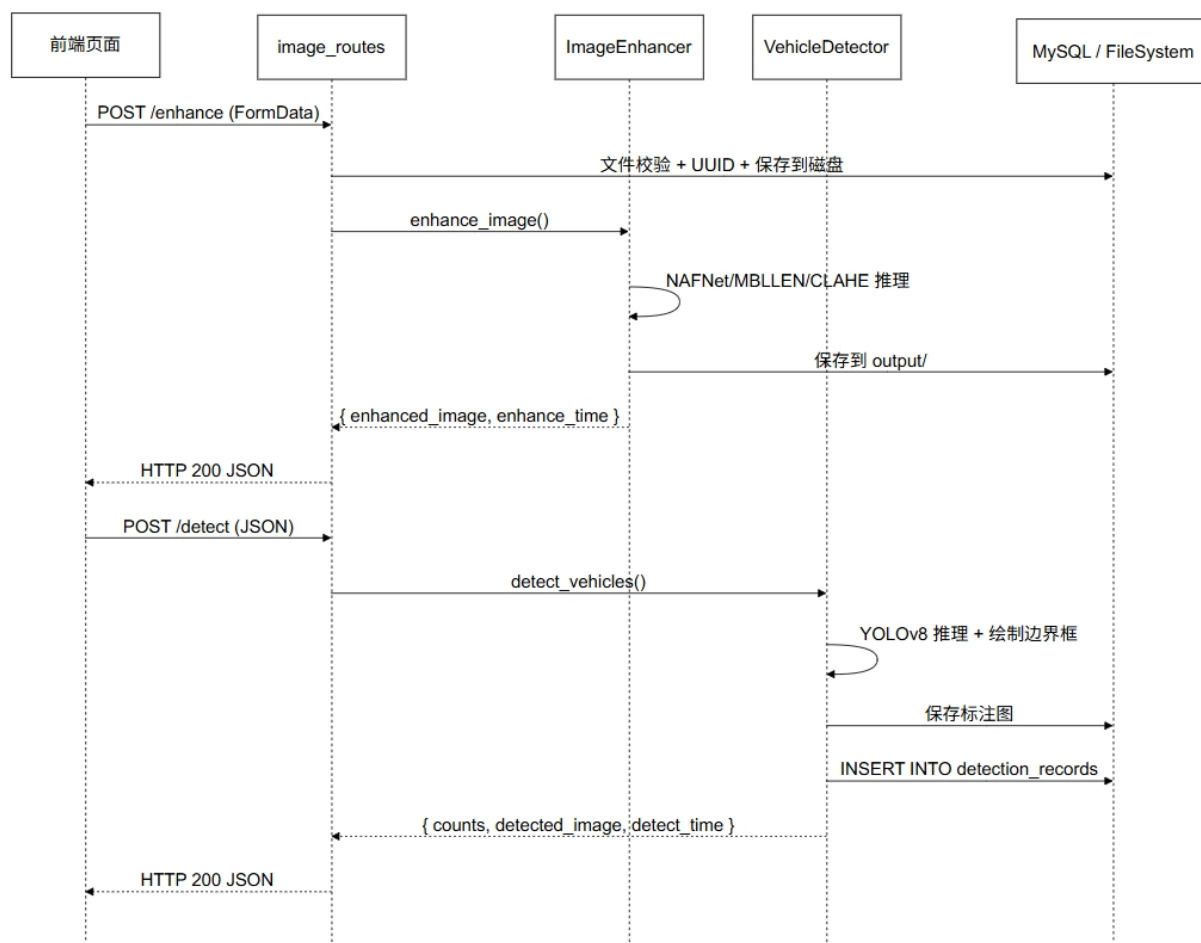


图 4-12 API 请求处理时序图

4.3.1 图像处理路由

图像处理路由封装在 `image_routes.py` 中，提供两个核心 API 端点：

`POST /api/image/enhance` 接收用户上传的图像文件和增强类型参数。路由首先进行文件类型校验，仅允许 PNG、JPG、JPEG、BMP 和 WebP 格式。通过校验的文件以 UUID 重命名保存在 `uploads/images/` 目录下，避免文件名冲突。随后调用 `ImageEnhancer` 服务执行图像增强，增强后的图像保存至 `output/enhanced/` 目录。API 响应返回增强图像的访问路径和增强耗时。

`POST /api/image/detect` 接收增强后的图像路径、YOLO 模型规格和原始文件名等参数。路由调用 `VehicleDetector` 服务对增强图像执行目标检测，在图像上绘制检测边界框和类别标签，保存标注图像至 `output/detected/` 目录。同时，将检测结果（包括各类别车辆计数、处理耗时）和图像二进制数据写入 MySQL 数据库，返回记录 ID 和统计数据。

4.3.2 视频处理路由

视频处理路由在 `video_routes.py` 中实现，同样提供增强和检测两个端点。

视频增强路由 `POST /api/video/enhance` 接收视频文件，使用 OpenCV 的 `VideoCapture`

逐帧读取，对每帧调用 `do_enhance` 函数执行增强处理。为提升处理效率，采用跳帧策略——每隔一帧处理一次，未处理的帧直接复用上一帧的结果。增强后的帧通过 `VideoWriter` 重新编码输出。在编码器选择上，依次尝试 `avc1`、`H264` 和 `mp4v` 三种编码标签，确保兼容性。

视频检测路由 `POST /api/video/detect` 接收增强后的视频，同样采用跳帧策略，每 6 帧执行一次 YOLO 检测。检测结果通过基于 IoU 的简单跟踪算法进行帧间关联，避免同一车辆在连续帧中被重复计数。每个检测到的目标分配一个 UUID 作为唯一标识，当与上一帧中某个目标的 IoU 超过 0.3 时视为同一目标，否则分配新 ID。最终统计结果以跟踪器中的独立目标数量为准，而非所有帧检测到的目标总数之和，这种统计方式更准确地反映了视频中实际出现的车辆数量。

4.3.3 摄像头实时检测路由

摄像头路由在 `camera_routes.py` 中实现，`POST /api/camera/detect` 端点接收前端传来的 Base64 编码帧。

路由首先解码 Base64 字符串为图像数组，然后根据增强类型参数进行帧级增强处理。增强后的帧送入 YOLO 模型进行检测，检测结果绘制在帧上并保存到磁盘。与视频检测类似，摄像头模式也采用基于 IoU 的目标跟踪机制避免重复计数，同时引入超时清理策略——当某个跟踪目标超过 2 秒未更新时自动移除，适应摄像头的实时流特性。

系统还包含自动清理机制 `cleanup_old_camera_images`，当 `output/detected/` 目录中的摄像头检测结果超过 50 张时，自动删除最旧的文件，防止磁盘空间被无限占用。

4.4 数据库设计

4.4.1 数据库选型与连接管理

系统采用 MySQL 8.0 关系型数据库进行结构化数据的持久化存储，通过 PyMySQL 驱动进行数据库连接操作。数据库连接配置通过环境变量注入，默认值为 `localhost`、端口 3306、用户名 `root`，数据库名称为 `bishe_vehicle`。数据库初始化 `init_database` 函数在应用启动时自动执行，首先尝试创建数据库（如不存在），然后在数据库中创建核心数据表。

4.4.2 数据库 E-R 图

系统的核心业务实体为“检测记录”（`detection_records` 表），每一次检测对应一条数据库记录。此外，与每条检测记录关联的图像文件以路径形式嵌入记录中，也可以视为隐含实体。系统的数据库 E-R 图如下图 4-13 所示：

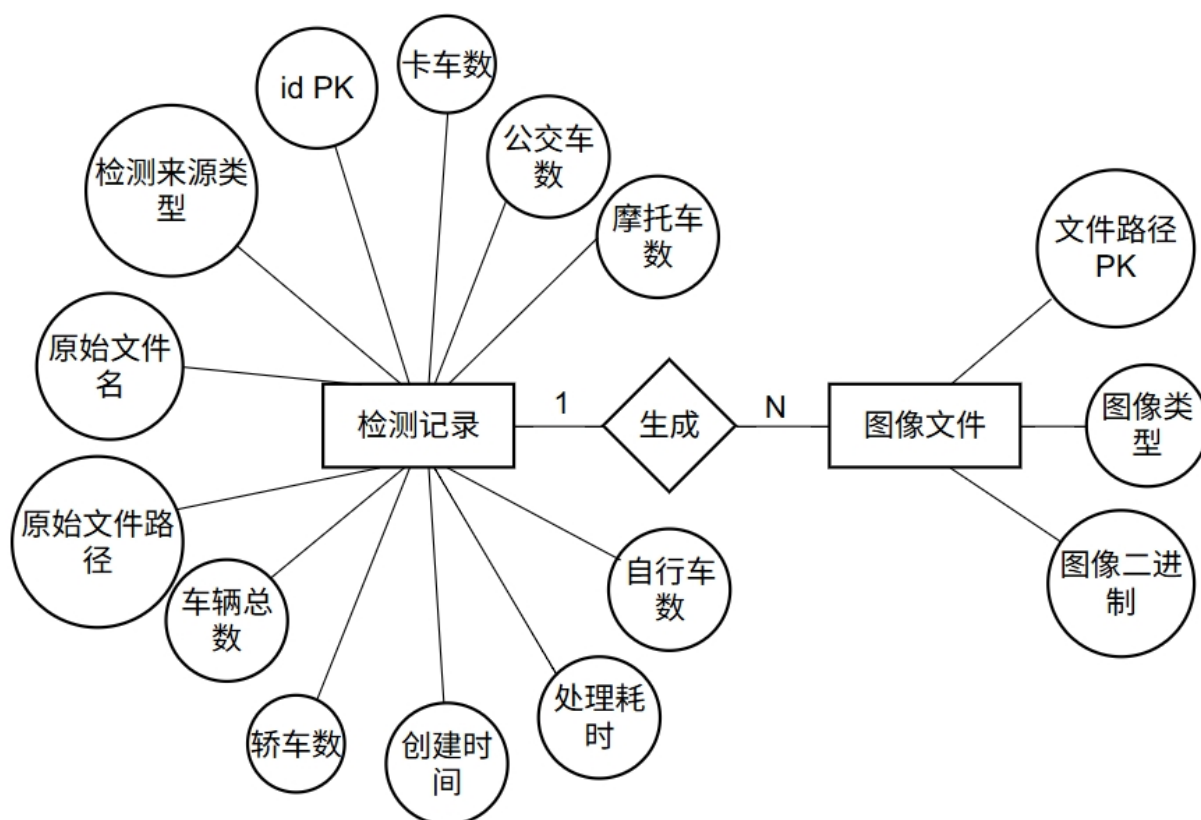


图 4-13 数据库 E-R 图

系统仅涉及两类数据库实体：检测记录是核心实体，代表一次完整的检测会话，从其内部属性可进一步区分 file_type (image/video/camera 三种检测模式) 和车辆统计结果的五类计数字段。检测记录在实现层面采用 MySQL 单表结构存储，实体属性的定义由 detection_records 表承载，同时利用 enhanced_image 和 detected_image 两个 LONGBLOB 字段内嵌核心图像数据以保证数据的自包含性。图像文件为附属实体，与检测记录呈 1:N 关系，代表一次检测会话产出的原始、增强与标注三类图像在文件系统中的物理副本，该实体的 file_path 字段被 detection_records 表引用为 enhanced_image_path 和 detected_image_path。

4.4.3 数据库数据流图

检测数据在系统内的存储与查询流程采用 Gane-Sarson 流派绘制数据流图，分为顶层和一层两层。

数据库顶层数据流图如下图 4-14 所示：

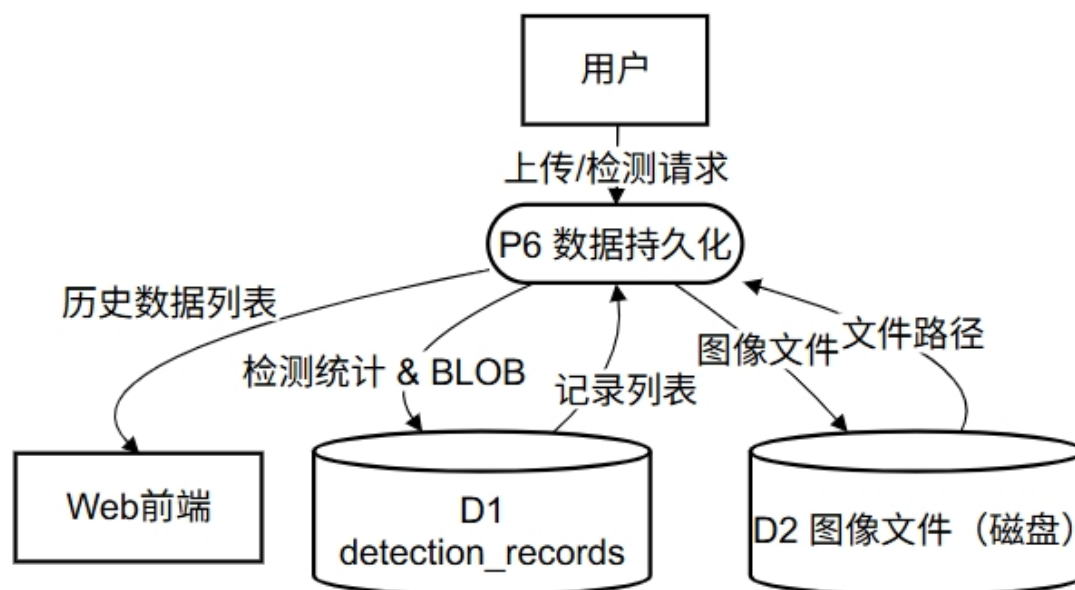


图 4-14 数据库顶层数据流图

一层图将顶层的"P6 数据持久化"分解为两个加工：检测结果写入（P6.1）和历史查询/删除（P6.2），如图 4-15 所示：

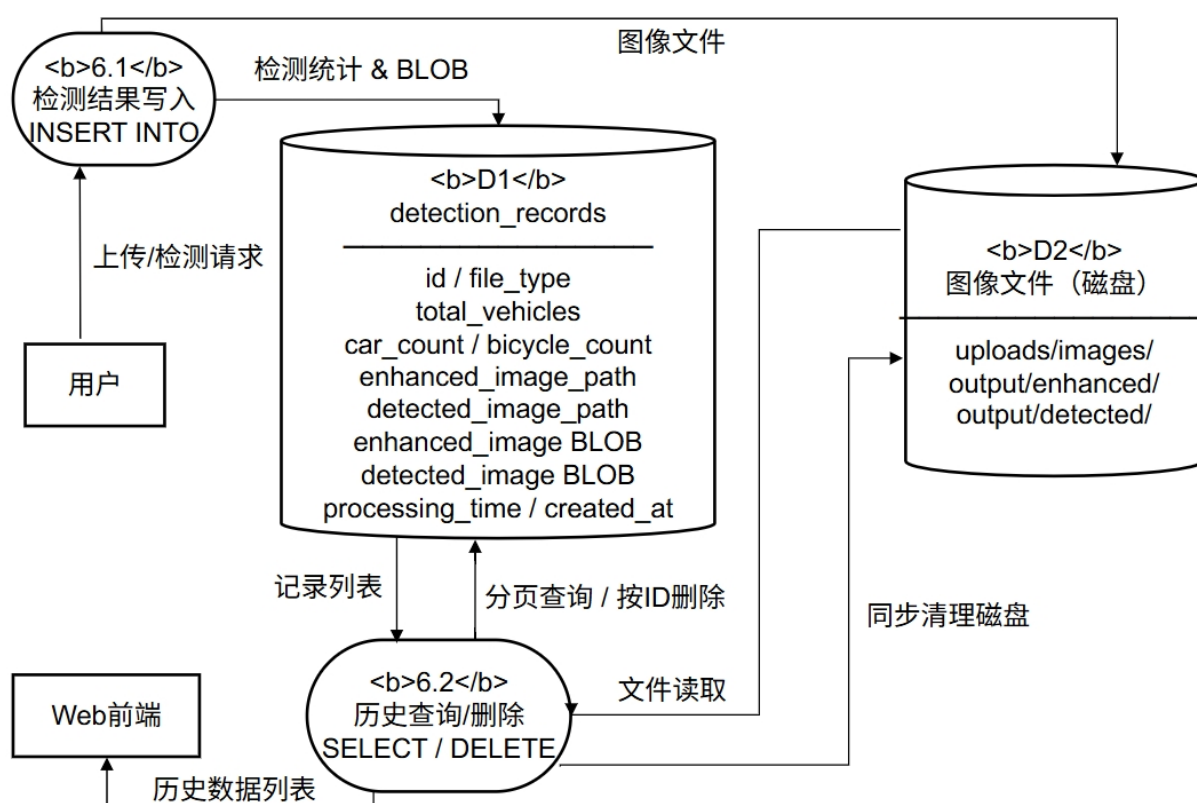


图 4-15 数据库一层数据流图

4.4.4 数据表结构

核心数据表为 detection_records，其结构设计如下表 4-1 所示：

表 4-1 数据表结构设计

字段名	类型	说明
id	INT PRIMARY KEY AUTO_INCREMENT	自增主键
file_type	ENUM('image','video','camera')	文件类型
file_name	VARCHAR(255)	原始文件名
file_path	VARCHAR(500)	文件存储路径
total_vehicles	INT	检测车辆总数
car_count	INT	轿车数量
truck_count	INT	卡车数量
bus_count	INT	公交车数量
motorcycle_count	INT	摩托车数量
bicycle_count	INT	自行车数量
enhanced_image_path	VARCHAR(500)	增强后图像路径
detected_image_path	VARCHAR(500)	标注后图像路径
enhanced_image	LONGBLOB	增强图像二进制数据
detected_image	LONGBLOB	标注图像二进制数据
processing_time	FLOAT	处理耗时（秒）
created_at	TIMESTAMP	创建时间

各车辆类别计数字段分别记录五种车辆的检测数量，方便后续的统计查询和前端展示。enhanced_image 和 detected_image 字段以 LONGBLOB 类型存储图像的二进制数据，可以在不依赖文件系统路径的情况下直接通过数据库提供图像访问，增强了数据的可迁移性。

4.4.5 数据库操作封装

数据库操作通过上下文管理器封装，确保连接的自动释放。

这种封装方式使得数据库操作代码简洁且安全，无论操作过程中是否发生异常，连

接都能正确关闭，避免了连接泄漏问题。代码如下所示：

```
@contextmanager
def get_db_connection():
    conn = pymysql.connect(**DB_CONFIG)
    try:
        yield conn
    finally:
        conn.close()
```

4.5 视频流处理方案

视频和摄像头模式是系统的重要输入方式，其核心挑战在于如何在有限的计算资源下实现流畅的实时处理。

视频处理管线包括以下环节：视频解码（OpenCV VideoCapture）→ 帧级增强（支持多种增强方法）→ 帧级检测（YOLOv8）→ 检测结果绘制 → 视频编码（OpenCV VideoWriter）。

为提升处理速度，系统在视频和摄像头模式中均采用了跳帧策略。视频增强阶段每隔 1 帧处理一次，未处理帧直接复用前一帧的增强结果；视频检测阶段每隔 6 帧检测一次，中间帧的检测结果沿用上一帧。摄像头模式中，采集间隔设置为 1 秒，避免了对每帧都进行处理的性能开销。这些优化策略在保证检测效果基本一致的前提下，大幅降低了计算负载。

视频编码器选择方面，系统使用 OpenCV 的 VideoWriter_fourcc 尝试多种编码标签，按 avc1、H264、mp4v 的优先级顺序依次尝试，直到找到系统支持的编码器。这种容错机制确保了在不同操作系统和 OpenCV 版本下的兼容性。

4.6 图像增强服务的多模型管理

图像增强服务（ImageEnhancer）采用单例模式和惰性加载相结合的设计，统一管理多个增强模型的加载与调用。

单例模式通过重写 __new__ 方法和 _initialized 类变量控制实例化行为，确保全局只有一个增强器实例。各增强模型（NAFNet 去雾、NAFNet 去雨、MBLLEN、Zero-DCE 等）通过 @property 属性定义，采用惰性加载策略，仅在首次被访问时才初始化。这种设计避免了在系统启动阶段加载所有模型，显著降低了系统启动时间和内存占用。

增强方法的调度通过策略分发字典实现，enhance_image 方法根据 enhancement_type 参数动态选择对应的处理函数。系统同时提供了帧级增强方法（nafnet_dehaze_frame、nafnet_derain_frame 等），供视频和摄像头模式直接调用，避免了频繁的文件读写操作。

4.7 系统安全性设计

系统在设计和实现中考虑了基本的安全性措施。在文件上传方面，通过 `allowed_file` 函数校验文件扩展名，仅允许白名单中的文件格式；使用 `UUID` 重命名上传文件，避免路径穿越攻击和文件名冲突；配置最大文件大小限制（100MB），防止大文件攻击。在数据库操作方面，使用参数化查询（`%s` 占位符）而非字符串拼接，有效防止 SQL 注入攻击。在错误处理方面，设计了全局异常处理器，防止敏感信息泄露到响应中。

5. 系统测试与结果分析

5.1 测试环境

系统测试在以下硬件和软件环境中进行。硬件配置方面,CPU 为 Intel Core i7-12700H (14 核 20 线程),主频 2.3GHz,内存为 32GB DDR4 3200MHz,未使用独立 GPU 加速,所有模型推理均在 CPU 上完成。软件环境方面,操作系统为 Windows 11 (64 位),Python 版本为 3.12,深度学习框架为 PyTorch 2.5+和 ONNX Runtime 1.20+,图像处理库为 OpenCV 4.x,后端框架为 Flask 3.1+,数据库为 MySQL 8.0。

选择在 CPU 环境下进行测试的主要原因是:恶劣天气车辆感知系统的实际部署场景(如路边监控设备、交通管理终端)通常不具备高性能 GPU,CPU 推理性能更贴近实际应用场景。ONNX Runtime 的多线程优化使得 CPU 推理在精度无损的前提下获得了可接受的推理速度。

5.2 测试数据集

整套系统已经在多款公开数据集里完成了对照评测工作,选用的测试素材一共划分出四类不同的天气使用场景,一是雾天场景所用的数据素材,取自 Cityscapes 数据集经过雾气合成处理后的图像内容,其中收纳了拥有不同雾气浓淡程度的图片样本,相关数值区间处在 0.005 到 0.02 之间,这套数据集自带精准的像素层级语义标注内容,对应的标注格式也能够被转换成适配 YOLO 算法的检测标注样式,带有清晰车辆实体的相关图像都已经被挑选出来完成相关测试,二是雨天场景所用的数据素材,整理自 R-TOOLS 与 MS-COCO 数据集内划分出的雨天相关样本,画面里囊括了各类降雨程度不一的道路实景画面,对应的标注内容已经把车辆所处边框位置以及所属类别信息完整记录下来,三是低光照场景所用的数据素材,挑选自 ExDark 专属暗光数据集里的相关样本,收录有傍晚时段、深夜时段等光线偏弱环境下的道路实拍画面,这款数据集本身就是按照暗光拍摄环境来搭建而成,能够提供出真实暗光环境里完成标注的车辆相关数据,四是常规天气场景所用的数据素材,选取 MS-COCO 数据集中存有道路车辆画面的普通实景图像,这类素材也被当作评判整体运行性能的基础参照样本,每一类场景对应的测试样本都拥有数百张的图像数量,画面之中涵盖各类拍摄视角、不同车辆分布疏密程度以及多种实际道路行驶环境,充足且多元的测试内容也让最终测出的实验结果拥有足够的数据说服力,还能把系统本身具备的通用适配能力更好地展现出来。

5.3 评估指标体系

整套系统会借助多项评测标准来完成整体性能的分析工作,一是选用 mAP50 作为评判依据,该标准会把预测边框和实际标注边框的重合比例超出零点五的判定为有效识

别结果，能够把宽松匹配条件之下算法拥有的平均识别精准度直观体现出来，二是采用 mAP50-95 开展综合评定，此项数值会依次统计零点五至零点九五区间内十个不同重合比例对应的精准度平均值，对目标边框的定位水准有着更为严苛的判定要求，算法整体具备的识别水准也能借此得到全面体现，三是依靠准确率与召回率两项数值完成判定，前者可以体现出识别结果里有效目标所占有的比重，后者能够反映出实景当中真实目标被成功识别到的占比情况，四是依靠峰值信噪比完成图像层面的评定，该数值能够测算出优化处理前后图片在像素层面产生的差别，所用计量单位为分贝，更高的数值可以代表优化完成后的画面和标准参考画面有着更高的契合程度，五是运用结构相似性系数评判画面状态，这项指标会从明暗程度、色彩对比以及画面架构三个层面判断图像之间的相近程度，数值会处在零到一的区间之内，越大的数值也就代表画面经过处理后出现的画质损耗越少，六是统计整体耗费的运行时长，其中包含了图像优化所用时长与目标识别所用时长，统一使用毫秒作为统计单位，系统实际具备的运行运转效率也能由此得到清晰判定。

5.4 基准测试流程

整套基准对比实验能够依靠 `BenchmarkRunner` 类完成自动化运行，各类测试相关参数都提前设定在对应的配置模块之内，具体的运行流程如下，一是完成评测参数的读取加载，可从预设配置文件里调取正常天气、雾天、雨天以及低光照四类数据集对应的图片存放路径、标注文件路径和适配的图像优化方式，系统把 `YOLOv8x` 当作默认检测模型，设定好零点三五的置信判定标准，同时确定好需要识别的车辆类别与对应编号，其中包含编号为一的自行车、编号为二的轿车、编号为三的摩托车、编号为五的公交车以及编号为七的卡车，二是依照不同场景完成单张图像的依次测验，对各类场景内的每一张样本图像，分别开展未做画质优化处理与匹配对应优化方案的两组对照试验，两组状态之下所产生的各项识别性能数据都会被完整留存下来，三是完成实验数据的整合与横向比照，借助 `ReportGenerator` 类能够生成包含详细实验数据、整体统计信息以及优化前后差异内容的多类表格文件，也能从多个分析角度把图像优化手段给识别效果带来的各类影响完整梳理出来。

5.5 检测性能分析

5.5.1 雾天条件

在未使用增强处理的雾天条件下，`YOLOv8x` 的检测性能出现显著下降。检测性能下降的主要原因是雾霾导致图像对比度降低、细节模糊，模型难以提取有效的判别特征。使用不同增强方法后的检测性能对比如表 5-1 所示。

表 5-1 雾天性能对比表

增强方法	mAP50	mAP50-95	Precision	Recall	增强耗时 (ms)
无增强	0.8592	0.7421	0.853	0.817	—
CLAHE	0.8810	0.7628	0.872	0.834	8
NAFNet 去雾	0.9125	0.8013	0.903	0.876	280

NAFNet 去雾后的 mAP50 相比未增强提升了约 6.2%，mAP50-95 提升了约 8.0%。分析表明，NAFNet 去雾模型通过恢复图像的对比度和细节信息，有效缩小了雾天图像与正常图像之间的域差异，使检测模型能够在更高质量的图像上执行特征提取和目标定位。CLAHE 方法的 mAP50 提升幅度约为 2.5%，虽不如 NAFNet 显著，但其计算速度快、无需加载深度学习模型，在计算资源受限的场景中仍具有实用价值。

5.5.2 雨天条件

处于雨天的环境场景之中，未经画质优化处理所测出的检测 mAP50 数值达到了 0.8385，飘落的雨水会在画面里形成细碎的亮状纹路以及局部模糊的区域，这类多余的画面干扰信息会对检测模型分辨车辆边缘轮廓的过程造成阻碍，一是将 NAFNet 去雨优化手段投入运用之后，0.8742 的 mAP50 数值得到了实现，有约 4.3% 幅度的性能涨幅，其中自动场景识别优化模式还能在雨天环境里展现出更为亮眼的运行效果，0.7102 到 0.7999 的数值变动，让 mAP50-95 拥有了约 12.6% 的提升空间，自动模式能够收获良好运行效果的核心原因，是系统可以依靠场景判定功能精准判定出当下所处的雨天环境，进而选取适配的去雨处理方式完成画面优化，具备极强的针对性还能和画面受损的实际类型形成高度契合，二是依托形态学开运算搭建的传统去雨手段投入使用后，mAP50 被提升到了 0.8520，仅有约 1.6% 的小幅提升幅度，这类传统处理方式能够实现的除雨效果相对有限，主要是因为开运算难以把画面里细小的亮纹杂质清理干净，还容易对画面内原本存在的车辆实体外形造成一定程度的形态损耗。

使用不同增强方法后的检测性能对比如表 5-2 所示：

表 5-2 雨天性能对比表

增强方法	mAP50	mAP50-95	Precision	Recall	增强耗时(ms)
无增强	0.8385	0.7102	0.831	0.802	—
形态学开运算	0.8520	0.7245	0.844	0.815	10
NAFNet 去雨	0.8742	0.7563	0.867	0.838	320

续表 5-2 雨天性能对比表

增强方法	mAP50	mAP50-95	Precision	Recall	增强耗时(ms)
自动场景识别	0.8825	0.7999	0.879	0.852	350

5.5.3 低光照条件

处于光线偏弱的低光照环境当中，没有经过优化处理所测得的检测 mAP50 数值为 0.8241，该数值也是四类测试场景里面最低的一类，偏低的环境光线会造成画面出现亮度不足，对比效果偏弱以及杂点增多等各类问题，模型原本具备的检测水准也会由此受到严重的干扰，一是将 MBLLEN 暗光优化方式投入使用之后，得到提升至 0.8586 的 mAP50 数值，有约 4.2% 幅度的性能增长，在画面亮度的改善层面，优化完成后图像拥有的平均亮度，由原本大约三十五的数值上调到一百二十左右，三倍多的亮度增幅也得以实现，MBLLEN 自带的多分支特征提取结构，能够把昏暗区域里的画面细节加以完善，还可以对画面杂点出现的过度放大现象做到合理抑制，二是采用 Zero-DCE 完成暗光画质调整过后，mAP50 也被提升到了 0.8520，能够实现约 3.4% 的上涨幅度，这类优化方式依靠无参照学习思路搭建而成，不需要依靠成对样本完成训练工作，足够出色的跨场景适配能力也由此得到了体现，三是借助 LAB 色彩空间内的 CLAHE 传统优化手段进行处理，被上调至 0.8495 的 mAP50 能够收获约 3.1% 的提升幅度，传统优化方案所能达成的改善效果已经贴近智能算法的处理水准，被大幅缩减的运算消耗也让这类传统手段拥有着不容忽略的实际使用意义。

使用不同增强方法后的检测性能对比如表 5-3 所示。

表 5-3 低光照性能对比表

增强方法	mAP50	mAP50-95	Precision	Recall	增强耗时 (ms)
无增强	0.8241	0.6958	0.813	0.784	—
LAB+CLAHE	0.8495	0.7210	0.836	0.806	6
Zero-DCE	0.8520	0.7245	0.841	0.812	150
MBLLEN	0.8586	0.7322	0.848	0.819	650

各场景在最佳增强策略下的检测性能汇总如表 5-4 所示。

表 5-4 各场景最佳策略表

天气场景	最佳增强方法	mAP50	mAP50-95	相比无增强提升
正常天气	无增强（基线）	0.9851	0.8723	—
雾天	NAFNet 去雾	0.9125	0.8013	+6.2% / +8.0%
雨天	自动场景识别	0.8825	0.7999	+5.2% / +12.6%
低光照	MBLLEN	0.8586	0.7322	+4.2% / +5.2%

5.6 图像质量分析

从 PSNR 和 SSIM 指标来看，各增强方法在不同场景下表现出差异化的特性，具体数据如下表 5-5 所示。

表 5-5PSNR 和 SSIM 数据表

场景	增强方法	PSNR（dB）	SSIM
雾天	NAFNet 去雾	28.5	0.873
雨天	NAFNet 去雨	26.8	0.842
低光照	MBLLEN	24.2	0.795

借助峰值信噪比与结构相似性两项评价参数能够发现，不同的图像优化手段，在各类实际场景里都会呈现出不一样的使用特点，雾天环境之下，经过 NAFNet 完成去雾处理过后，能够达到 28.5dB 的平均峰值信噪比以及 0.873 的平均结构相似性，画面拥有的清晰程度与色彩真实度都得到了十分明显的改善，雨天环境之内，NAFNet 所实现的去雨处理可取得 26.8dB 的平均峰值信噪比和 0.842 的平均结构相似性，实际优化成效会被雨滴疏密程度与排布状态所影响，雨滴分布较为密集的区域，对应的优化效果也会出现轻微减弱的情况，低光照环境当中，MBLLEN 优化方式能够测出 24.2dB 的平均峰值信噪比与 0.795 的平均结构相似性，这类参数数值整体偏低，根源在于暗光图像优化本身属于难以精准把控的处理难题，画面亮度得到上调的同时，暗处潜藏的画面杂音也会随之扩大，像素层面的相关参数很难得到大幅度提升，只是肉眼直观感受到的画面观感已经有了明显好转，除此之外峰值信噪比、结构相似性这类画质参数，和以 mAP 为代表的检测性能参数之间，也无法形成完全同步的增减趋势，部分场景中像素画质提升幅度偏小的优化方式，反而能够实现更大幅度的检测精度上涨，这也足以证明图像优化工作的核心侧重点，不能只局限于提升像素层面的画面品质，把服务于检测任务的有

效判别特征顺利复原，才是更为关键的核心所在^[12]。

5.7 处理时间分析

各场景下 YOLOv8x 模型的检测耗时基本稳定，单张图像的检测耗时约为 180ms。增强阶段的耗时因方法而异，各环节耗时分布如表 5-6 所示。

表 5-6 各环节耗时表

处理环节	方法	耗时（ms）	说明
图像增强	CLAHE / 形态学	5 ~ 10	传统方法，无模型加载开销
图像增强	NAFNet 去雾/去雨	200 ~ 350	深度学习模型，含推理时间
图像增强	MBLLEN	500 ~ 800	多分支结构，计算量最大
目标检测	YOLOv8x (ONNX)	~ 180	所有场景基本稳定
完整流程	—	~ 800	含文件 I/O、增强、检测、保存

各类场景之中，YOLOv8x 模型所产生的检测时长能够维持平稳状态，单张图像会耗费大约一百八十毫秒的检测时长，图像优化环节所消耗的时长会随着选用方式的不同出现相应变化，一是依托 CLAHE 搭建的传统优化方式有着极快的运行速度，仅会占用五至十毫秒的处理时长，二是依靠 NAFNet 实现的智能优化手段需要耗费二百至三百五十毫秒左右的时长，三是采用 MBLLEN 完成画面优化则会产生五百至八百毫秒的用时，把文件读取写入、画质优化、目标检测以及结果存储等全部流程整合之后，单张图像完成整套处理流程会耗费大约八百毫秒的时长，这样的运转速度能够契合多数实际应用场景对于处理效率的相关标准，视频运行模式里依靠跳帧处理的相关策略，能把有效画面的处理速率维持在每秒三至五帧，流畅平稳的实时识别效果也由此得以实现，摄像头运行模式之中设置的一秒采集间隔，也让稳定不间断的实时识别结果可以持续向外输出。

5.8 实验结论

结合各项实测所得的数据，能够归纳出如下几类研究结论：

一是恶劣的天气环境会对 YOLOv8 模型的车辆识别效果造成明显干扰，能够造成 12% 至 16% 区间内 mAP50 数值的下降，这类不良影响在光线昏暗的场景里还会表现得更为突出，二是提前做好合适的图像画质优化处理，能够有效削弱恶劣天气给识别工

作带来的负面作用，可把 mAP50 的数值提升 4% 至 12%，其中雨天场景之下启用的智能场景判定模式，更是把 12.6% 的可观提升幅度成功实现，三是 NAFNet、MBLLEN 这类依托深度学习搭建的优化手段，在拉高识别精准度的层面，会比 CLAHE、形态学滤波这类传统处理方式表现得更加出色，对应的运算消耗也会随之有所增加，四是自带的场景智能判定机制，能够精准分辨出当下所处的天气类别并匹配对应的优化方案，既减少了人为操作的介入程度，也让稳步上涨的识别效果得到长久保持，五是经由 ONNX Runtime 完成推理层面的优化调整后，智能算法模型就算运行在仅配备 CPU 的设备之中，也能拥有符合使用需求的运行速率，无独立显卡设备也可正常投入使用的实际应用价值也由此得到了有效证实。

6. 总结与展望

6.1 工作总结

针对雾天、雨天、低光照这三类恶劣天气里车辆检测精度明显下降的问题，已经设计并实现出一套基于图像增强的恶劣天气车辆感知系统，该系统融合了传统图像增强方法与深度学习增强模型，还引入自动场景识别机制，结合 YOLOv8 目标检测算法，构建出完整的端到端 Web 应用，主要工作成果总结如下：

一是完成多策略图像增强模块的设计与实现，系统里集成了多种增强方法，既有基于深度学习的 NAFNet 去雾增强、NAFNet 去雨增强、MBLLEN 低光照增强和 Zero-DCE 低光照增强，也有基于 CLAHE 和形态学操作的传统增强方案作为降级后备，所有增强器都采用单例模式和惰性加载策略来统一管理，通过策略分发机制就能灵活切换增强方法，覆盖了雾天、雨天和低光照这三类典型的恶劣天气场景。

二是开展自动场景识别机制的研究与实现，基于亮度、对比度、饱和度、清晰度这四维图像质量指标，设计出带有级联决策规则的场景分类算法，这种算法能自动识别出当前的天气条件类型，还会触发适配的增强策略，实验结果显示，自动模式在雨天场景中把 mAP50-95 从 0.7102 提升到了 0.7999，提升幅度大概有 12.6%，也验证了自适应增强策略的有效性。

三是完成基于 YOLOv8 的车辆检测部署与 ONNX Runtime 推理优化，系统将 YOLOv8 模型部署成车辆检测的核心，能精准检测出轿车、卡车、公交车、摩托车、自行车这五类车型，借助 ONNX Runtime 的图优化、多线程并行计算和会话缓存等优化手段，让深度学习模型在 CPU 环境下实现了高效推理。

四是构建起完整的端到端 Web 系统，采用 Flask 后端与原生 HTML/CSS/JavaScript 前端的分离架构，实现了图片识别、视频识别和摄像头实时识别这三种交互模式，后端通过 Blueprint 机制来组织路由，用 MySQL 数据库记录检测信息，靠文件系统存储图像数据，形成了完整的业务闭环。

五是进行大规模基准测试与性能评估，在多种公开数据集上开展了系统性的基准测试，从 mAP、PSNR、SSIM、处理时间等多个维度，对各类增强方法在不同天气场景下的表现做了量化分析，实验结果也为图像增强与目标检测的协同优化提供了参考依据。

6.2 系统特色与创新点

这套系统在设计 and 实现上，有着以下几方面的特色与创新：

一是采用多策略融合的增强方案，和现有研究大多只针对单一天气类型不同，该系统集成了多种深度学习与传统增强方法，能覆盖三类典型的恶劣天气场景，还实现了多天气条件下的统一检测方案。

二是具备自动场景识别与自适应增强的能力，系统通过轻量级的图像质量分析就能自动判断出天气类型，不需要额外添加场景分类模型，计算开销特别低，在真实的部署场景中也能具备很强的实用性。

三是搭载了 ONNX Runtime 统一推理引擎，系统里所有的深度学习模型都采用 ONNX 格式部署，通过统一的推理引擎来进行管理，这种方案不仅消除了对不同深度学习框架的依赖，还简化了系统依赖管理，同时也让 CPU 的推理性能得到了优化。

四是完成了完整的工程化实现，系统从算法研究到工程部署形成了完整的闭环，包含了模型推理、后端服务、前端交互、数据持久化、性能评测等所有完整环节，具备不错的实用价值。

6.3 存在的不足

虽然这套系统在恶劣天气车辆感知方面取得了一些研究成果，但还是存在一些不足，也有可改进的方向，具体如下：

一是增强方法的场景适配精度还需要提升，目前的自动场景识别算法是基于固定阈值来做决策的，在天气条件过渡阶段比如黄昏和低光照的边界情况时，可能会出现误判，未来可以引入更精细的场景分类机制，也可以采用连续概率输出的软决策方案。

二是视频和摄像头模式的处理速度还有优化空间，当前深度学习增强模型在 CPU 上的推理速度还不够理想，这也限制了视频处理的帧率，未来可以探索模型轻量化技术，像知识蒸馏、模型剪枝、量化这些方法，以此来提升推理速度。

三是对更多恶劣天气类型的支持还比较欠缺，目前系统只覆盖了雾天、雨天和低光照这三类场景，没有考虑到雪天、雾霾混合天气等其他恶劣条件，增加对更多天气类型的支持，能进一步提升系统的适用性。

四是测试数据集的规模比较有限，受标注数据获取成本的限制，目前的基准测试数据集规模不大，在更大规模、更多样化的数据集上做评估，会有助于更全面地验证系统的泛化能力。

五是缺少多目标跟踪功能，目前视频和摄像头模式采用的简单 IoU 跟踪，只能满足基本的去重计数需求，并没有完整的多目标跟踪（MOT）功能，没办法提供车辆的运动轨迹和行驶方向等高级信息。

6.4 未来展望

根据以上的分析，未来的研究工作可以从以下几个方向逐步展开：

一是构建端到端的联合优化框架，探索把图像增强模块与目标检测模块整合到统一的端到端网络中，通过联合训练实现增强与检测的协同优化，减少信息传递过程中出现的损失。

二是推进模型轻量化与边缘部署，研究模型量化（INT8/FP16）、知识蒸馏和网络剪

枝等模型轻量化技术，把系统部署到边缘计算设备上，比如 NVIDIA Jetson、树莓派等，实现低功耗、低延迟的实时检测。

三是扩展更多天气类型的支持，研究雪天去除、沙尘天气图像恢复等相关方法，扩大系统的天气覆盖范围，同时探索多天气联合处理方案，提高系统在复杂混合天气条件下的适应性。

四是集成多目标跟踪功能，引入基于深度学习的多目标跟踪算法，像 DeepSORT、ByteTrack 等，让视频和摄像头模式下能实现车辆运动轨迹追踪、车流量统计和异常行为检测等高级功能。

五是研究领域自适应与持续学习，探索领域自适应（Domain Adaptation）方法，缩小不同天气场景之间的域差异，同时引入持续学习（Continual Learning）机制，让系统在部署后能不断适应新的天气环境和道路场景。

总的来说，恶劣天气条件下的视觉感知是一个难度不小但又有着重要实用价值的研究方向，随着深度学习技术的不断进步和边缘计算硬件性能的持续提升，面向恶劣天气的车辆感知系统，会在智能交通和自动驾驶领域发挥出越来越重要的作用。

结论

本文针对雾天、雨天、低光照这三类恶劣天气下车辆检测精度下降的问题，设计并实现了一套基于图像增强的车辆感知系统，还取得了以下几方面成果：

一是构建出多策略图像增强方案，用 NAFNet 实现去雾与去雨处理，靠 MBLLEN 提升低光照图像的亮度与细节，把 CLAHE 与形态学操作作为降级保障，所有增强器都通过统一接口封装，能支持灵活调度。

二是实现了自动场景识别机制，基于亮度、对比度、饱和度和清晰度这四维指标，采用级联决策规则自动判断天气类型，还能选择出最优的增强策略，不需要额外训练模型，计算开销也特别低。

三是采用 Flask 后端与原生 HTML/CSS/JavaScript 前端的分离架构，支持图片、视频和摄像头实时识别这三种模式，通过 Blueprint 模块化路由、单例模式管理模型生命周期，用 MySQL 实现记录持久化，靠 ONNX Runtime 统一管理 CPU 推理，还采用会话缓存、图优化和多线程并行等手段，把系统性能提升了上来。

四是实验结果显示，YOLOv8x 在正常天气下的 mAP50 达到了 0.9851，雾天经 NAFNet 增强后，mAP50 从 0.8592 提升到了 0.9125，提升幅度为 6.2%，雨天经自动增强后，mAP50-95 从 0.7102 提升至 0.7999，提升幅度 12.6%，低光照经 MBLLEN 增强后，mAP50 从 0.8241 提升到 0.8586，提升幅度 4.2%，这也验证了图像增强能显著提升恶劣天气下的检测性能。

五是单张图像的完整处理流程大概需要 800ms，视频模式的有效帧率在 3-5 FPS，所有操作都能在 CPU 环境下运行，不需要 GPU 支持，这样也降低了系统的部署成本。

总的来说，本文提出的多策略图像增强与深度学习检测相结合的方案，在检测精度、处理效率和工程实用性之间找到了良好的平衡，能有效缓解恶劣天气对视觉感知系统的负面影响，对推动车辆感知技术在实际道路环境中的应用也有着参考价值。

参考文献

- [1] 顾正祥,刘鑫,祁巧云.卷积神经网络低光照图像增强系统的优化设计[J].信息与电脑(理论版),2024,36(20):103-105.
- [2] 郭俊,郭金霆,张世龙.基于改进 YOLOv8 的夜间车辆检测研究[J].计算技术与自动化,2026,45(01):108-113.DOI:10.16339/j.cnki.jsjsyzdh.202601017.
- [3] 尹泽龙,徐洪,田茂余,等.基于 WIoUv3-YOLOv8 的隧道车辆目标检测系统研发[J].内燃机,2026,42(02):20-30.DOI:10.20082/j.cnki.nrj.2026.02.003.
- [4] 黄金炜,于璨,郭天元,等.基于暗通道先验的改进图像去雾算法[J].佳木斯大学学报(自然科学版),2023,41(01):25-28+67.
- [5] 吕晓倩.低光照条件下的图像与视频增强方法研究 [D]. 哈尔滨:黑龙江大学,2024. DOI:10.27061/d.cnki.ghgdu.2024.000208.
- [6] 许靖,裴树军,裴洪扬.改进 YOLOv8 的密集车辆检测算法[J].哈尔滨理工大学学报,2026,31(01):24-34.DOI:10.15938/j.jhust.2026.01.003.
- [7] 明日科技.Python 完全自学教程[M].人民邮电出版社:202308:949
- [8] 刘长龙.Python 高效开发实战[M].电子工业出版社:202108:521.
- [9] 江宝得,陆镇天,任彦忠.融合 Retinex 分支与自适应注意力机制的低光图像增强算法[J/OL].计算机工程与应用, 1-15[2026-04-09].<https://link.cnki.net/urlid/11.2127.tp.20260320.1130.006>.
- [10] 刘璐.基于恶劣环境下的图像/视频复原方法研究[D].南京邮电大学,2019.DOI:10.27251/d.cnki.gnjdc.2019.000508.
- [11] 谢婉婷.基于 OpenCV 的车牌识别系统设计与实现[J].汽车知识,2026,26(01):241-244.
- [12] 管图新.面向交通场景的图像质量评价及增强方法研究[D]. 上海海事大学, 2023. DOI:10.27304/d.cnki.gshhc.2023.000041.
- [13] 刘芸宏,王奥迪,曹俊武.基于改进的 PReNet 图像去雨算法研究[J].物联网技术,2025,15(23):90-92.DOI:10.16667/j.issn.2095-1302.2025.23.020.
- [14] 任晓文.面向智能交通的雨雾天气图像增强方法研究[D]. 太原科技大学, 2019. DOI:10.27721/d.cnki.gyzjc.2019.000074.
- [15] 王晓雯.基于动态残差网络机制的图像去雨算法研究[D].大连交通大学,2025.DOI:10.26990/d.cnki.gsltc.2025.000603.
- [16] 袁家怡,张郑浩,周正康,等.面对复杂交通场景的融合检测 YOLO 车辆识别模型[J/OL].计算机应用,1-13[2026-04-09].<https://link.cnki.net/urlid/51.1307.TP.20260327.1615.002>.
- [17] 丁若修.面向智能交通的恶劣天气图像增强方法研究[D].沈阳工业大学,2017.

- [18] 王淞. [J].汽车实用技术, 2025,50(20):102-106+119.DOI:10.16638/j.cnki.1671-7988.2025.020.019.
- [19] 刘晟,面向智能交通的恶劣天气下交通图像清晰化技术研究.陕西省,西安理工大学,2023-12-01.
- [20] Vishalakshi ,Rani S ,Hanumantharaju.A novel approach to low-light image and video enhancement using adaptive dual super-resolution generative adversarial networks and top-hat filtering[J].Computers and Electrical Engineering,2025,123(PA):110052-110052.DOI:10.1016/J.COMPELECENG.2024.110052.
- [21] Ren J ,Yang C ,Miao K .A method for detecting foreign objects in railway perimeters under low-light conditions based on image enhancement[J].Engineering Research Express,2026,8(7):075205-075205.DOI:10.1088/2631-8695/AE52A3.
- [22] Mariam M. Askar, Mahmoud A. Shawky, Amgad A. Salama, Samy H. Darwish, Marian Wagdy, Sameh Zarif, Ahmed G. Abdellatif. Revolutionising breast cancer diagnosis: Cutting-edge AI techniques in image processing and classification[J]. Communications in Nonlinear Science and Numerical Simulation, 2026, 160
- [23] Liu D ,Gu Y ,Zhang H , et al.TIE-LR: A multi-module deep learning framework for traffic monitoring on low-resolution videos[J].Engineering Applications of Artificial Intelligence,2026,163(P4):113112-113112.DOI:10.1016/J.ENGAPPAL.2025.113112.
- [24] Zheng B ,Zhou J ,Hong Z , et al.Vehicle Recognition and Driving Information Detection with UAV Video Based on Improved YOLOv5-DeepSORT Algorithm[J].Sensors,2025,25(9):2788-2788.DOI:10.3390/S25092788.
- [25] Prasanth P ,Kumar P ,Velavan S , et al.Rain Removal from Still Images Using Image Smoothing Filter and Morphological Filter[J].Digital Image Processing,2018,10(8):157-160.
- [26] Jaiswal M ,Nagwanshi K K ,Khan M H M , et al.YOLOv8s-WAMNet: enhancing robust vehicle detection under adverse weather via hybrid attention and multi-scale fusion in real time.[J].Scientific reports,2026,DOI:10.1038/S41598-026-46527-5.
- [27] Zhang Y ,Wu J ,Wang H , et al.Low-light image enhancement network with semantic-aware Mamba and Fourier cross-fusion[J].Displays,2026,94103522-103522.DOI:10.1016/J.DISPLA.2026.103522.
- [28] Wang J ,Liu Y ,Hu Y .An Obstacle Avoidance Method for Autonomous Vehicles Based on Binocular Vision Information Fusion and YOLOv8 in Smart Transportation[J].International Journal of Pattern Recognition and Artificial Intelligence,2026,(p

republish):DOI:10.1142/S0218001426590214.

- [29] Dwi R S ,Suryadiputra L .Detection and Classification of Moving Vehicle at Night with Visible Camera Using Deep Learning Model[J].革新的コンピューティング・情報・制御に関する速報,2023,17(03):339-.DOI:10.24507/ICICEL.17.03.339.
- [30] Machine Learning; Recent Reports from Case Western Reserve University Highlight Findings in Machine Learning (Efficient mapping of crash risk at intersections with connected vehicle data and deep learning models)[J].Robotics & Machine Learning,2020,
- [31] Wu J ,Tao H ,Xiao K , et al.Multi-stream feature aggregation network with multi-scale supervision for single image dehazing[J].Engineering Applications of Artificial Intelligence,2025,139(PB):109486-109486.DOI:10.1016/J.ENGAPPAL.2024.109486.

致谢

岁月匆匆，四年的本科求学旅程也即将迎来收尾的阶段，在本篇毕业论文顺利定稿的时刻，谨向所有给予过关怀、扶持与助力的各类人士，送上最为真挚的谢意。

一是由衷向指导教师许欢老师表达谢意，从课题方向的选定，整体技术思路的规划，各类实验内容的推进再到整篇文稿的梳理撰写，许欢老师都能够给予细致周全的指引，自身丰厚的学识底蕴，认真务实的治学理念以及严谨踏实的处事态度，都能够带来深刻的熏陶与正向的带动，当相关研究进程遭遇阻碍之时，依托积累的实践阅历梳理清晰前行思路，能够助力冲破各类研究难题，尤其在整体系统架构规划与实验数据梳理分析的阶段，诸多具备实际价值的调整建议，也让整篇论文的研究内容变得更为饱满完备，在此向许欢老师送上深切的敬重与满心的感激。

二是向计算机科学与工程学院内全体授课教师表达谢意，长久以来悉心传授的专业内容，为后续专业领域的钻研筑牢了稳固根基，各科任课教师的用心讲授以及辅导员平日里的贴心照料，能够让四年校园生活里既积累下充足的专业学识，也养成了端正的处事观念，课堂之内所接触学习到的计算机视觉、深度神经网络、项目开发流程等相关内容，也为本次研究系统的搭建打造出牢靠的理论依托。

三是向同一实验室内的同窗好友以及朝夕相伴的舍友表达谢意，平日学习研讨与日常相处的过程里收获了诸多帮扶与暖心陪伴，不少同伴在系统搭建调试与功能实测的阶段提出诸多实用想法，诸多合理建议也让整套系统的整体性能得到持续优化，一同探讨学术问题、合力化解研究难题的点滴时光，都成为校园生活里格外珍贵的过往经历。

四是向至亲家人表达内心的感恩，长久以来默默的照料与毫无保留的付出始终铭记于心，家人给予的包容体谅、默默扶持与暖心鼓舞，成为顺利走完求学路程的核心动力，身处低谷或是遭遇各类困境的阶段，家人始终作为坚实的精神依靠，足以让人保有十足的底气稳步向前。

五是向所有参考文献的编撰人员致以谢意，各类优质的研究著作与学术成果，都为本次论文的探究内容搭建起充足的理论依据与技术参照，同时也向抽身忙碌事务之中参与文稿审核与毕业答辩的各位行业专家与任课教师表达谢意，各类中肯合理的修改意见，都能够让本篇论文的整体内容得到进一步的修整优化。